

Práctica PaaS: Despliegue de Servicio GenAI en Azure App Service (Rama Mínimo)

Este documento detalla la estructura, generación de imagen y flujo de comandos para realizar el despliegue del servicio mínimo en **Azure App Service (PaaS)** sobre Linux. El servicio consiste en un único contenedor que expone una API desarrollada en **FastAPI** que conecta con la API de **Gemini** para procesar prompts de usuario.

Componentes Azure Necesarios

Para llevar a cabo el despliegue de esta arquitectura en la nube, se requieren los siguientes componentes de **Microsoft Azure**:

- **Grupo de Recursos (Resource Group):** Contenedor lógico que agrupa todos los recursos relacionados con este despliegue para facilitar su administración y eliminación conjunta.
- **Azure Container Registry (ACR):** Registro privado de Docker en Azure donde se compilan, almacenan y gestionan las imágenes de los contenedores (**backend** y **frontend**).
- **Plan de App Service (App Service Plan):** Define los recursos físicos de computación y el sistema operativo (Linux con SKU B1) sobre los que correrá la aplicación.
- **Azure App Service (Web App for Containers):** Servicio PaaS que ejecuta la aplicación web a partir de la configuración multi-contenedor definida en el archivo **docker-compose.yml**.
- **Cuenta de Almacenamiento (Storage Account):** Servicio que aloja y gestiona las capacidades de almacenamiento en la nube requeridas para el proyecto.
- **Azure Files (File Share):** Recurso compartido de archivos montado como volumen persistente en la Web App (mapeado a **/app/data**) para salvaguardar el archivo de logs **history.txt** y asegurar su persistencia tras reinicios de los contenedores.

URLs del Servicio Desplegado en Azure

i Nota: Todos los bloques de este documento comparten la misma Web App de Azure (**webapp-fastapi-gemini**), actualizándose iterativamente. La URL base es única y permanece constante a través de las distintas versiones desplegadas.

Endpoint	URL	Descripción
Generación de contenido	https://webapp-fastapi-gemini.azurewebsites.net/generate?prompt=...	Envía un prompt a Gemini y devuelve la respuesta
Historial de logs	https://webapp-fastapi-gemini.azurewebsites.net/logs	Devuelve el historial de comunicaciones Frontend→Backend (disponible desde Bloque 4)

Descripción de los Despliegues

La práctica implementa un servicio de Inteligencia Artificial Generativa (GenAI) basado en la API de **Google Gemini**, desplegado de forma iterativa en **Azure App Service (PaaS)** a través de cuatro bloques de complejidad creciente:

Bloque	Tipo	Descripción resumida
1	Contenedor único	Una sola imagen FastAPI que recibe prompts del usuario y los envía directamente a la API de Gemini. Sin persistencia, sin proxy. Se despliega en Azure App Service (Linux) usando Azure Container Registry (ACR).
2	Docker Compose — 2 contenedores	Se divide en un Frontend (gateway público en puerto 80) y un Backend (procesador Gemini en puerto 8080, sin exposición directa al exterior). El Frontend enruta las peticiones al Backend internamente. Despliegue multi-contenedor en Azure App Service.
3	Compose + Persistencia (backend)	Sobre el Bloque 2, el Backend guarda cada prompt en un archivo history.txt montado sobre Azure Files (File Share). El historial sobrevive a reinicios y reempliegues gracias al volumen persistente.
4	Compose + Persistencia (frontend) + Logs	El Frontend pasa a ser el responsable de registrar logs de comunicación (URL llamada, respuesta recibida) en history.txt . El volumen persistente se traslada al Frontend. Se añade el endpoint /logs para consultar el historial. Es la arquitectura final desplegada.

Todos los bloques utilizan imágenes propias construidas desde cero (no se usa DockerHub) y almacenadas en el **Azure Container Registry** planservicioia.azurecr.io.

Prerrequisitos del Entorno

Antes de comenzar a ejecutar los comandos de despliegue o pruebas locales, asegúrate de contar con los siguientes elementos instalados y configurados en tu equipo:

1. Docker Desktop

- **Uso:** Construcción, etiquetado y ejecución de contenedores locales, así como publicación de imágenes en el registro privado.
- **Instalación:** Descarga e instala [Docker Desktop para Windows](#).
- **Verificación:** Ejecuta el siguiente comando en PowerShell para verificar que el servicio está activo:

```
docker info
```

```
PS C:\Users\danie\azure-assignment> docker info
Client:
Version:      28.5.1
Context:      desktop-linux
Debug Mode:   false
Plugins:
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
   Version:   v1.9.11
   Path:      C:\Program Files\Docker\cli-plugins\docker-ai.exe
buildx: Docker Buildx (Docker Inc.)
   Version:   v0.29.1-desktop.1
   Path:      C:\Program Files\Docker\cli-plugins\docker-buildx.exe
compose: Docker Compose (Docker Inc.)
   Version:   v2.40.3-desktop.1
   Path:      C:\Program Files\Docker\cli-plugins\docker-compose.exe
debug: Get a shell into any image or container (Docker Inc.)
   Version:   0.0.45
   Path:      C:\Program Files\Docker\cli-plugins\docker-debug.exe
desktop: Docker Desktop commands (Docker Inc.)
   Version:   v0.2.0
   Path:      C:\Program Files\Docker\cli-plugins\docker-desktop.exe
extension: Manages Docker extensions (Docker Inc.)
   Version:   v0.2.31
   Path:      C:\Program Files\Docker\cli-plugins\docker-extension.exe
init: Creates Docker-related starter files for your project (Docker Inc.)
   Version:   v1.4.0
   Path:      C:\Program Files\Docker\cli-plugins\docker-init.exe
mcp: Docker MCP Plugin (Docker Inc.)
   Version:   v0.26.0
   Path:      C:\Users\danie\.docker\cli-plugins\docker-mcp.exe
model: Docker Model Runner (Docker Inc.)
   Version:   v0.1.46
   Path:      C:\Program Files\Docker\cli-plugins\docker-model.exe
offload: Docker Offload (Docker Inc.)
   Version:   v0.5.10
   Path:      C:\Program Files\Docker\cli-plugins\docker-offload.exe
```

- **Inicio rápido (si no está activo):** Si Docker no se está ejecutando, puedes iniciarlo desde el menú de inicio de Windows o por PowerShell:

```
Start-Process "C:\Program Files\Docker\Docker\Docker Desktop.exe"
```

Espera a que el icono de Docker en la barra de tareas muestre "Docker Desktop is running" antes de continuar.

2. Azure CLI

- **Uso:** Administración, creación y configuración de toda la infraestructura PaaS (Azure App Service, Azure Container Registry, Azure Files).
- **Instalación:** Sigue la [guía de instalación de Azure CLI para Windows](#).

- **Verificación:** Ejecuta el siguiente comando en PowerShell para confirmar que está instalado correctamente:

```
az --version
```

```
PS C:\Users\danie\azure-assignment> az --version
azure-cli                        2.79.0 *
core                            2.79.0 *
telemetry                        1.1.0

Extensions:
account                          0.2.5

Dependencies:
msal                             1.34.0b1
azure-mgmt-resource              23.3.0

Python location 'C:\Program Files\Microsoft SDKs\Azure\CLI2\python.exe'
Config directory 'C:\Users\danie\.azure'
Extensions directory 'C:\Users\danie\.azure\cliextensions'

Python (Windows) 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)]

Legal docs and information: aka.ms/AzureCliLegal

You have 2 update(s) available. Consider updating your CLI installation with 'az upgrade'
```

Bloque 1: Despliegue de un Contenedor

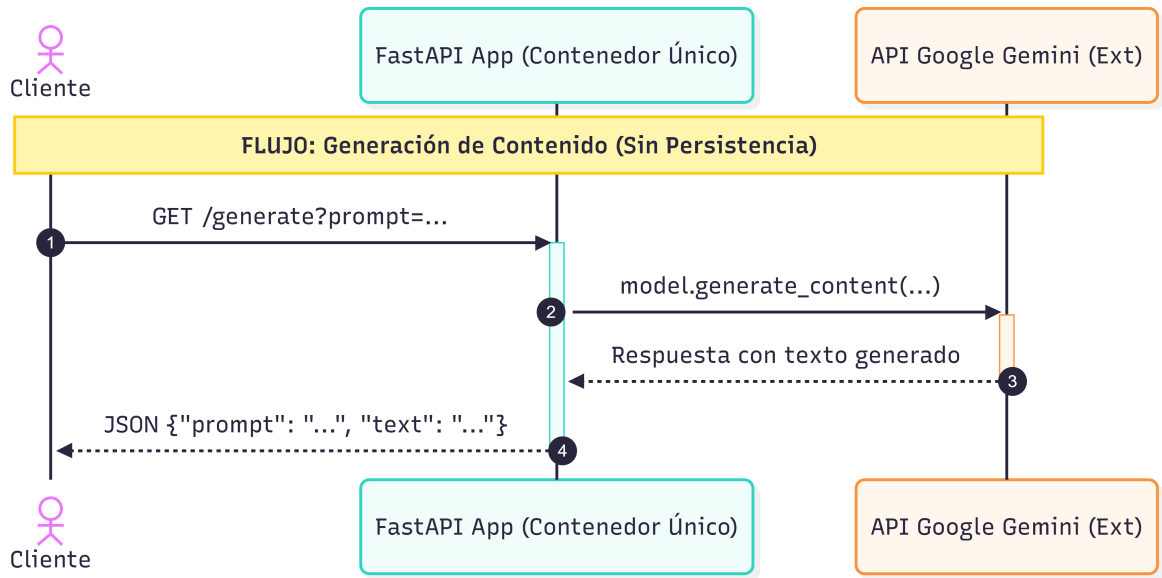
1.1. Descripción del Despliegue (Arquitectura Mínima)

 **Código de la primera versión:** El código fuente correspondiente a este Bloque 1 está disponible en el [Release v1 de GitHub](#).

El despliegue consiste en un servicio web gestionado en contenedor dentro de la plataforma **Azure App Service (Linux Web App)**.

Características del despliegue:

- **Sin Persistencia:** El contenedor funciona bajo el principio de infraestructura efímera (stateless). Las solicitudes se procesan en tiempo real y no se guarda historial de las consultas en archivos locales ni bases de datos.
- **Único Contenedor:** Toda la lógica reside en una sola aplicación de FastAPI, eliminando la necesidad de proxies o bases de datos adicionales.
- **Integración GenAI:** Se conecta directamente con la API oficial de Google Gemini usando el SDK de nueva generación (`google-generativeai`) y la clave de API `GEMINI_API_KEY` inyectada mediante variables de entorno en la nube.
- **Infraestructura PaaS:** Se emplea un plan de App Service en Linux, ideal para albergar APIs basadas en contenedores con escalado automático de recursos.



1.2. Definición del Contenedor (Dockerfile)

La imagen se construye a partir del siguiente **Dockerfile**, el cual optimiza el peso de la imagen mediante la versión **slim** de Python y aprovecha la caché de capas de Docker copiando primero las dependencias.

```
# Use official Python slim image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements.txt first to leverage Docker cache
COPY requirements.txt .

# Install all dependencies from requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

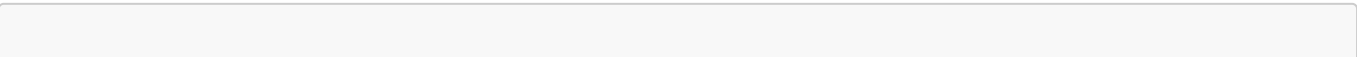
# Copy the FastAPI app
COPY app.py .

# Expose FastAPI port
EXPOSE 8080

# Run the app with Uvicorn
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Código de la Aplicación (app.py):

El servicio expone un único endpoint **/generate** que interactúa de manera asíncrona con el modelo **gemini-2.5-flash**:



```
# app_fastapi_prompt.py
from dotenv import load_dotenv
from fastapi import FastAPI, Query, HTTPException
from google import genai
import os

# Carga variables de entorno locales (si existen)
load_dotenv()
client = genai.Client()

app = FastAPI()

@app.get("/generate")
async def generate_content(prompt: str = Query(..., description="The prompt to send to Gemini")):
    try:
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=prompt,
        )
        return {"prompt": prompt, "text": response.text}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

1.3. Pruebas y Validación en Local

Antes de proceder con el despliegue en la nube, se recomienda validar el correcto funcionamiento de la aplicación de manera local mediante Docker.

⚠ Prerequisito: Asegúrate de que **Docker Desktop** está arrancado antes de ejecutar cualquier comando **docker**. Puedes iniciarlo desde el menú de inicio de Windows o ejecutando **Start-Process "C:\Program Files\Docker\Docker\Docker Desktop.exe"** en PowerShell. Espera a que el icono de Docker en la barra de tareas muestre el mensaje *"Docker Desktop is running"* antes de continuar.

Ejecución mediante Docker local

Construye la imagen localmente y levanta el contenedor pasando la variable de entorno (para ejecución directa sin Docker, existe un archivo **.env** en la raíz del proyecto con la clave **GEMINI_API_KEY**):

```
# Leer la API Key desde el archivo .env (evita exponer la clave en el historial de comandos)
$env:GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" |
ForEach-Object { $_.ToString().Split("=",2)[1] })

# Construir la imagen local
docker build -t fastapi-gemini:local .

# Ejecutar el contenedor
```

```
docker run -d -p 8080:8080 --name fastapi-app-local -e
GEMINI_API_KEY=$env:GEMINI_API_KEY fastapi-gemini:local
```

```
PS C:\Users\danie\azure-copy> # Leer la API Key desde el archivo .env (evita exponer la clave en el historial de comandos)
>> $env:GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" | ForEach-Object { $_.ToString().Split("=",2)[1] })
>>
>> # Construir la imagen local
>> docker build -t fastapi-gemini:local .
>>
>> # Ejecutar el contenedor
>> docker run -d -p 8080:8080 --name fastapi-app-local -e GEMINI_API_KEY=$env:GEMINI_API_KEY fastapi-gemini:local
[+] Building 2.5s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 500B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:cdbd05fb6f457ca275ff51ce00d93d865ca0b6a25f5ffb08262d94f6835771e5
=> => resolve docker.io/library/python:3.11-slim@sha256:cdbd05fb6f457ca275ff51ce00d93d865ca0b6a25f5ffb08262d94f6835771e5
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:223cf9a960fc4d48715822e50212924d374b04f1517f1045566cf5423f607742
=> => exporting config sha256:f4e9b3a409522b2c0b6d580e89ff8611ff6a17d8c181a2435a7b81747aafd8c4
=> => exporting attestation manifest sha256:15b9beaeca5ebf2fef5f4132537450217a52f0d7101653d203636fae984aad77
=> => exporting manifest list sha256:e4510dd1806048c0b1093a4516413b64f7d701d3449910d5941173a435b727cf
=> => naming to docker.io/library/fastapi-gemini:local
=> => unpacking to docker.io/library/fastapi-gemini:local
81df9671b1b51bb625643c3dbe5fc5906b76faed4214752757fa385dc46a60ed
PS C:\Users\danie\azure-copy>
```

Verificación local:

Realiza una llamada de prueba desde el navegador o consola al puerto local **8080**:

```
Invoke-WebRequest "http://localhost:8080/generate?prompt=Hello_Local_Testing" -
UseBasicParsing
```

i Nota sobre PowerShell: Se añade el parámetro `-UseBasicParsing` en todos los comandos `Invoke-WebRequest` de esta práctica para evitar que PowerShell intente inicializar el motor de Internet Explorer (lo cual suele lanzar una advertencia de seguridad interactiva y requerir confirmación del usuario para continuar).

```
PS C:\Users\danie\azure-copy> Invoke-WebRequest "http://localhost:8080/generate?prompt=Hello_Local_Testing" -UseBasicParsing
>>

StatusCode      : 200
StatusDescription : OK
Content         : {"prompt":"Hello_Local_Testing","text":"Hello! Looks like you're doing some local testing. How can I assist you today?"}
RawContent      : HTTP/1.1 200 OK
                  Date: Sun, 28 Jun 2026 22:53:28 GMT
                  Server: uvicorn
                  Content-Length: 120
                  Content-Type: application/json

                  {"prompt":"Hello_Local_Testing","text":"Hello! Looks like you're doing som...
Headers         : {[Date, System.String[]], [Server, System.String[]], [Content-Length, System.String[]], [Content-Type, System.String[]]}
Images          : {}
InputFields     : {}
Links           : {}
RawContentLength : 120
RelationLink    : {}

PS C:\Users\danie\azure-copy>
```

1.4. Guía de Pasos Realizados (Despliegue en Azure)

A continuación se presenta la secuencia completa de comandos ejecutados en la consola para registrar la imagen de contenedor y aprovisionar la infraestructura en Azure.

Paso 1: Autenticación en Azure

Inicia sesión en tu suscripción de Azure mediante CLI. Al ejecutar este comando, se abrirá automáticamente un diálogo en el sistema operativo para que inicies sesión con tus credenciales de Azure. Cuando termines el proceso de autenticación, tendrás que seleccionar la suscripción. Pulsa "Enter" en la consola para continuar con el predeterminado.

```
# Autenticarse en Azure
az login
```

```
PS C:\Users\danie\azure-assignment> az login
Select the account you want to log in with. For more information on login with Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136

[Tenant and subscription selection]

No      Subscription name      Subscription ID      Tenant
-----
[1] *   Azure for Students   [REDACTED]          Universidade da Coruña

The default is marked with an *; the default tenant is 'Universidade da Coruña' and subscription is 'Azure for Students' [REDACTED]

Select a subscription and tenant (Type a number or Enter for no changes):

Tenant: Universidade da Coruña
Subscription: Azure for Students [REDACTED]

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://go.microsoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default.

PS C:\Users\danie\azure-assignment>
```

Paso 2: Definición de variables de entorno

Define las variables de entorno de trabajo en tu sesión de PowerShell para utilizarlas en los siguientes comandos:

```
# Definir variables de entorno para los comandos
$RESOURCE_GROUP = "gruporecursosia"
```

```

$LOCATION = "spaincentral"
$ACR_NAME = "planservicioia"
$APP_SERVICE_PLAN = "plan-fastapi-gemini"
$WEB_APP_NAME = "webapp-fastapi-gemini"
$IMAGE_TAG = "planservicioia.azurecr.io/fastapi-gemini:v1"
# Leer la API Key desde el archivo .env
$GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" | ForEach-Object { $_.ToString().Split("=",2)[1] })

```

```

PS C:\Users\danie\azure-assignment> # Definir variables de entorno para los comandos
>> $RESOURCE_GROUP = "gruporecursosia"
>> $LOCATION = "spaincentral"
>> $ACR_NAME = "planservicioia"
>> $APP_SERVICE_PLAN = "plan-fastapi-gemini"
>> $WEB_APP_NAME = "webapp-fastapi-gemini"
>> $IMAGE_TAG = "planservicioia.azurecr.io/fastapi-gemini:v1"
>> # Leer la API Key desde el archivo .env
>> $GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" | ForEach-Object { $_.ToString().Split("=",2)[1] })
PS C:\Users\danie\azure-assignment>

```

Paso 3: Creación del Grupo de Recursos y del Azure Container Registry (ACR)

Crea un grupo de recursos donde se alojarán todos los componentes y aprovisiona el registro de contenedores privado:

```

# Crear el grupo de recursos
az group create --name $RESOURCE_GROUP --location $LOCATION

```

```

PS C:\Users\danie\azure-assignment> az group create --name $RESOURCE_GROUP --location $LOCATION
{
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia",
  "location": "spaincentral",
  "managedBy": null,
  "name": "gruporecursosia",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
PS C:\Users\danie\azure-assignment>

```

```

# Crear el Azure Container Registry (SKU básico para desarrollo/pruebas)
az acr create --name $ACR_NAME --resource-group $RESOURCE_GROUP --sku Basic

```

```
PS C:\Users\danie> az acr create --name $ACR_NAME --resource-group $RESOURCE_GROUP --sku Basic
{
  "adminUserEnabled": false,
  "anonymousPullEnabled": false,
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "creationDate": "2026-06-01T21:42:08.509823+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia/providers/Microsoft.ContainerRegistry/registries/planservicioia",
  "identity": null,
  "location": "spaincentral",
  "loginServer": "planservicioia.azurecr.io",
  "metadataSearch": "Disabled",
  "name": "planservicioia",
  "networkRuleBypassOptions": "AzureServices",
  "networkRuleSet": null,
  "policies": {
    "azureAdAuthenticationAsArmPolicy": {
      "status": "enabled"
    },
    "exportPolicy": {
      "status": "enabled"
    },
    "quarantinePolicy": {
      "status": "disabled"
    },
    "retentionPolicy": {
      "days": 7,
      "lastUpdatedTime": "2026-06-01T21:42:25.671511+00:00",
      "status": "disabled"
    },
    "softDeletePolicy": {
      "lastUpdatedTime": "2026-06-01T21:42:25.671544+00:00",
      "retentionDays": 7,
      "status": "disabled"
    },
    "trustPolicy": {
      "status": "disabled",
      "type": "Notary"
    }
  },
  "privateEndpointConnections": [],
  "provisioningState": "Succeeded",
  "publicNetworkAccess": "Enabled",
  "resourceGroup": "gruporecursosia",
  "roleAssignmentMode": "LegacyRegistryPermissions",
  "sku": {
    "name": "Basic",
    "tier": "Basic"
  },
  "status": null,
  "systemData": {
    "createdAt": "2026-06-01T21:42:08.509823+00:00",
    "createdBy": "daniel.quintillan@udc.es",
    "createdByType": "User",
    "lastModifiedAt": "2026-06-01T21:42:08.509823+00:00",
    "lastModifiedBy": "daniel.quintillan@udc.es",
    "lastModifiedByType": "User"
  },
  "tags": {},
  "type": "Microsoft.ContainerRegistry/registries",
  "zoneRedundancy": "Disabled"
}
PS C:\Users\danie> |
```

Paso 4: Habilitar usuario administrador en ACR y autenticarse localmente

Habilita el acceso de administrador para obtener las credenciales de lectura/escritura de imágenes directamente desde Docker:

```
# Habilitar usuario admin del registro
az acr update -n $ACR_NAME --admin-enabled true
```

```
PS C:\Users\danie> az acr update -n $ACR_NAME --admin-enabled true
{
  "adminUserEnabled": true,
  "anonymousPullEnabled": false,
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "creationDate": "2026-06-01T21:42:08.509823+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia/providers/Microsoft.ContainerRegistry/registries/planservicioia",
  "identity": null,
  "location": "spaincentral",
  "loginServer": "planservicioia.azurecr.io",
  "metadataSearch": "Disabled",
  "name": "planservicioia",
  "networkRuleBypassOptions": "AzureServices",
  "networkRuleSet": null,
  "policies": {
    "azureAdAuthenticationAsArmPolicy": {
      "status": "enabled"
    },
    "exportPolicy": {
      "status": "enabled"
    },
    "quarantinePolicy": {
      "status": "disabled"
    },
    "retentionPolicy": {
      "days": 7,
      "lastUpdatedTime": "2026-06-01T21:42:25.671511+00:00",
      "status": "disabled"
    },
    "softDeletePolicy": {
      "lastUpdatedTime": "2026-06-01T21:42:25.671544+00:00",
      "retentionDays": 7,
      "status": "disabled"
    }
  },
  "trustPolicy": {
    "status": "disabled",
    "type": "Notary"
  }
},
  "privateEndpointConnections": [],
  "provisioningState": "Succeeded",
  "publicNetworkAccess": "Enabled",
  "resourceGroup": "gruporecursosia",
  "roleAssignmentMode": "LegacyRegistryPermissions",
  "sku": {
    "name": "Basic",
    "tier": "Basic"
  },
  "status": null,
  "systemData": {
    "createdAt": "2026-06-01T21:42:08.509823+00:00",
    "createdBy": "daniel.quintillan@udc.es",
    "createdByType": "User",
    "lastModifiedAt": "2026-06-01T22:11:14.087718+00:00",
    "lastModifiedBy": "daniel.quintillan@udc.es",
    "lastModifiedByType": "User"
  },
  "tags": {},
  "type": "Microsoft.ContainerRegistry/registries",
  "zoneRedundancy": "Disabled"
}
PS C:\Users\danie>
```

Obtener la contraseña de administrador

```
$ACR_PASSWORD = (az acr credential show --name $ACR_NAME --query
"passwords[0].value" -o tsv)
```

Iniciar sesión local en Docker con las credenciales de ACR (usando password-stdin por seguridad)

```
echo $ACR_PASSWORD | docker login "$ACR_NAME.azurecr.io" --username $ACR_NAME --
password-stdin
```

```
PS C:\Users\danie\azure-assignment> # Obtener la contraseña de administrador
>> $ACR_PASSWORD = (az acr credential show --name $ACR_NAME --query "passwords[0].value" -o tsv)
>>
>> # Iniciar sesión local en Docker con las credenciales de ACR (usando password-stdin por seguridad)
>> echo $ACR_PASSWORD | docker login "$ACR_NAME.azurecr.io" --username $ACR_NAME --password-stdin
Login Succeeded
PS C:\Users\danie\azure-assignment>
```

Paso 5: Construcción y subida de la imagen Docker

Construye la imagen Docker localmente etiquetándola con la URI del registro de Azure, y súbela al repositorio:

```
# Construir la imagen usando el Dockerfile de la raíz
docker build -t $IMAGE_TAG .
```

```
PS C:\Users\danie\azure-assignment> docker build -t $IMAGE_TAG .
[+] Building 40.4s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 480B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 2.4s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e382 7.8s
=> => resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e382 0.1s
=> => sha256:45006ceeeea9e2ca59a046b6f4ac9a212e4b27b3d8d6e7d348b03f0aacddac99 250B / 250B      0.2s
=> => sha256:797d495f2c68eb4664d918d39078ca04612d7ef47c57b5c784f7f6eedd42bf5 14.37MB / 14.37MB 2.4s
=> => sha256:8649771fee179d7c2590c94f533aaa5fceb70e36e25dec83672fe836d99577c5 1.29MB / 1.29MB 1.0s
=> => sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccace970 29.78MB / 29.78MB 3.8s
=> => extracting sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccace970 1.9s
=> => extracting sha256:8649771fee179d7c2590c94f533aaa5fceb70e36e25dec83672fe836d99577c5 0.3s
=> => extracting sha256:797d495f2c68eb4664d918d39078ca04612d7ef47c57b5c784f7f6eedd42bf5 1.6s
=> => extracting sha256:45006ceeeea9e2ca59a046b6f4ac9a212e4b27b3d8d6e7d348b03f0aacddac99 0.0s
=> [internal] load build context                                  0.1s
=> => transferring context: 750B                                     0.0s
=> [2/5] WORKDIR /app                                           0.6s
=> [3/5] COPY requirements.txt .                                0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt     23.1s
=> [5/5] COPY app.py .                                          0.1s
=> => exporting to image                                           6.0s
=> => exporting layers                                           3.4s
=> => exporting manifest sha256:2b73642c63558f8389f2c419bac2672ba67bb07bfea6363ec9a2bf181ba709ae 0.0s
=> => exporting config sha256:9f704624731f1017fa36cfcf3ee12f8dc7217f09211cc0609afb9701c0db80fa 0.0s
=> => exporting attestation manifest sha256:12fa9a979a002e6b9ea70cc549c80bab673b95a44b6865decdd8d4e12b2198fd 0.0s
=> => exporting manifest list sha256:58291f09ae9cc90f11613e925afc7113657a4f339fe9a32fc7a8c612bb384253 0.0s
=> => naming to planservicioia.azurecr.io/fastapi-gemini:v1      0.0s
=> => unpacking to planservicioia.azurecr.io/fastapi-gemini:v1   2.3s
PS C:\Users\danie\azure-assignment>
```

```
# Subir la imagen a Azure Container Registry
docker push $IMAGE_TAG
```

```
PS C:\Users\danie\azure-assignment> docker push $IMAGE_TAG
The push refers to repository [planservicioia.azurecr.io/fastapi-gemini]
949a5aa2f50a: Pushed
08f1375a4fa3: Pushed
797d495f2c68: Pushed
e285e1798bd9: Pushed
5b4d6ff92fc4: Pushed
8649771fee17: Pushed
5dbf852422f1: Pushed
97ff604ed1b0: Pushed
45006ceeeea9: Pushed
v1: digest: sha256:58291f09ae9cc90f11613e925afc7113657a4f339fe9a32fc7a8c612bb384253 size: 856
PS C:\Users\danie\azure-assignment>
```

Paso 6: Creación del Plan de App Service

Aprovisiona un plan de hospedaje en Azure App Service configurado en Linux:

```
# Crear el Plan de App Service (SKU B1 de Linux como nivel básico de desarrollo)
az appservice plan create --name $APP_SERVICE_PLAN --resource-group
$RESOURCE_GROUP --sku B1 --is-linux
```

```
PS C:\Users\danie\azure-assignment> az appservice plan create --name $APP_SERVICE_PLAN --resource-group $RESOURCE_GROUP --sku B1 --is-linux
{
  "asyncScalingEnabled": false,
  "elasticScaleEnabled": false,
  "extendedLocation": null,
  "freeOfferExpirationTime": "2026-12-01T22:39:08.233333",
  "geoRegion": "Spain Central",
  "hostingEnvironmentProfile": null,
  "hyperV": false,
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfdde/resourceGroups/gruporecursosia/providers/Microsoft.Web/serverfarms/plan-fastapi-gemini",
  "isSpot": false,
  "isXenon": false,
  "kind": "linux",
  "kubeEnvironmentProfile": null,
  "location": "spaincentral",
  "maximumElasticWorkerCount": 1,
  "maximumNumberOfWorkers": 0,
  "name": "plan-fastapi-gemini",
  "numberOfSites": 0,
  "numberOfWorkers": 1,
  "perSiteScaling": false,
  "provisioningState": "Succeeded",
  "reserved": true,
  "resourceGroup": "gruporecursosia",
  "sku": {
    "capabilities": null,
    "capacity": 1,
    "family": "B",
    "locations": null,
    "name": "B1",
    "size": "B1",
    "skuCapacity": null,
    "tier": "Basic"
  },
  "spotExpirationTime": null,
  "status": "Ready",
  "subscription": "884c87e5-7027-435d-905b-1722c9bcfdde",
  "tags": null,
  "targetWorkerCount": 0,
  "targetWorkerSizeId": 0,
  "type": "Microsoft.Web/serverfarms",
  "workerTierName": null,
  "zoneRedundant": false
}
```

Paso 7: Despliegue de la Web App desde el ACR

Aprovisiona la Web App de contenedores configurando la imagen previamente cargada en el registro de Azure:

```
# Crear la Web App con la imagen del contenedor de Azure
az webapp create --resource-group $RESOURCE_GROUP --plan $APP_SERVICE_PLAN --name
$WEB_APP_NAME --container-image-name $IMAGE_TAG
```

```
PS C:\Users\danie\azure-copy> az webapp create --resource-group $RESOURCE_GROUP --plan $APP_SERVICE_PLAN --name $WEB_APP_NAME --container-image-name $IMAGE_TAG
vnet_route_all_enabled is not a known attribute of class <class 'azure.mgmt.web.v2024_11_01.models._models_py3.Site'> and will be ignored
{
  "autoGeneratedDomainNameLabelScope": null,
  "availabilityState": "Normal",
  "clientAffinityEnabled": true,
  "clientAffinityPartitioningEnabled": null,
  "clientAffinityProxyEnabled": false,
  "clientCertEnabled": false,
  "clientCertExclusionPaths": null,
  "clientCertMode": "Required",
  "cloningInfo": null,
  "containerSize": 0,
  "customDomainVerificationId": "1FE448438D76EB3CD09F2F8CDC94FD32281C423F2A99F0C044D6FA3298A1AE47",
  "dailyMemoryTimeQuota": 0,
  "daprConfig": null,
  "defaultHostName": "webapp-fastapi-gemini.azurewebsites.net",
  "dnsConfiguration": {
    "dnsAltServer": null,
    "dnsLegacySortOrder": null,
    "dnsMaxCacheTimeout": null,
    "dnsRetryAttemptCount": null,
    "dnsRetryAttemptTimeout": null,
    "dnsServers": null
  },
  "enabled": true,
  "enabledHostNames": [
    "webapp-fastapi-gemini.azurewebsites.net",
    "webapp-fastapi-gemini.scm.azurewebsites.net"
  ],
  "endToEndEncryptionEnabled": false,
  "extendedLocation": null,
  "ftpPublishingUrl": "https://waws-prod-esc-005.ftp.azurewebsites.windows.net/site/wwwroot",
  "functionAppConfig": null,
  "hostNameSslStates": [
    {
      "certificateResourceId": null,
      "hostType": "Standard",
      "ipBasedSslResult": null,
      "ipBasedSslState": "NotConfigured",
      "name": "webapp-fastapi-gemini.azurewebsites.net",
      "sslState": "Disabled",
      "thumbprint": null,
      "toUpdate": null,
      "toUpdateIpBasedSsl": null,
      "virtualIPv6": null,
      "virtualIp": null
    },
    {
      "certificateResourceId": null,
      "hostType": "Repository",
      "ipBasedSslResult": null,
      "ipBasedSslState": "NotConfigured",

```

Paso 8: Configuración de Credenciales de Registro y Variables de Entorno

Configura la autenticación del App Service contra el ACR para que pueda descargar las actualizaciones de la imagen y define los secretos de la aplicación:

```
# Vincular las credenciales del registro privado a la Web App (usando parámetros actualizados)
az webapp config container set --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP --container-image-name $IMAGE_TAG --container-registry-url "https://$ACR_NAME.azurecr.io" --container-registry-user $ACR_NAME --container-registry-password $ACR_PASSWORD

# Definir la API Key de Gemini y forzar el puerto 8080 dentro del contenedor
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --settings WEBSITES_PORT=8080 GEMINI_API_KEY=$GEMINI_API_KEY
```

```

PS C:\Users\daniel\azure-copy> # Vincular las credenciales del registro privado a la Web App (usando parámetros actualizados)
>> az webapp config container set --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP --container-image-name $IMAGE_TAG --container-registry-url "https://$ACR_NAME.azurecr.io" --container-registry-user $ACR_NAME --container-registry-password $ACR_PASSWORD
>>
>> # Definir la API Key de Gemini y forzar el puerto 8080 dentro del contenedor
>> az webapp config appsettings set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --settings WEBSITES_PORT=8080 GEMINI_API_KEY=$GEMINI_API_KEY
{
  {
    "name": "WEBSITES_ENABLE_APP_SERVICE_STORAGE",
    "slotSetting": false,
    "value": "false"
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_URL",
    "slotSetting": false,
    "value": "https://planservicioia.azurecr.io"
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_USERNAME",
    "slotSetting": false,
    "value": "planservicioia"
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_PASSWORD",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_CUSTOM_IMAGE_NAME",
    "value": "DOCKER|planservicioia.azurecr.io/fastapi-gemini:v1"
  }
}
{
  {
    "name": "WEBSITES_ENABLE_APP_SERVICE_STORAGE",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITES_PORT",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "GEMINI_API_KEY",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_URL",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_USERNAME",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_PASSWORD",
    "slotSetting": false,
    "value": null
  }
}
}
PS C:\Users\daniel\azure-copy>

```

1.5. Verificación del Funcionamiento en Azure

Una vez que el despliegue haya completado su inicio en Azure, puedes probar el endpoint de generación ejecutando la siguiente llamada HTTP desde la consola:

```

# Realizar petición de prueba a la API desplegada en Azure
Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?
prompt=Explain_PaaS_in_one_sentence" -UseBasicParsing

Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/generate?
prompt=Explain_PaaS_in_one_sentence" -UseBasicParsing

```

Formato esperado de respuesta:

```

{
  "prompt": "Explain_PaaS_in_one_sentence",
  "text": "Platform as a Service (PaaS) is a cloud computing model where a third-party provider delivers hardware and software tools—usually those needed for application development—to users over the internet."
}

```

```
PS C:\Users\danie\azure-copy> Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/generate?prompt=Explain_PaaS_in_one_sentence" -UseBasicParsing

StatusCode      : 200
StatusDescription : OK
Content         : {"prompt":"Explain_PaaS_in_one_sentence","text":"PaaS provides a cloud-based platform and tools for developers to build, deploy, and manage applications, abstracting away the underlying infrastructure..."}
RawContent      : HTTP/1.1 200 OK
                  Date: Sun, 28 Jun 2026 23:49:11 GMT
                  Server: uvicorn
                  Set-Cookie: ARRAffinity=c8fcc30ceb9d725a7f379f8837c91f9e4afa35242bd4fb5dcad86baef02d41ad;Path=/;HttpOnly;Secure;Domain=webapp-fas...
Headers         : {[Date, System.String[]], [Server, System.String[]], [Set-Cookie, System.String[]], [Content-Length, System.String[]]-}
Images          : {}
InputFields     : {}
Links           : {}
RawContentLength : 203
RelationLink    : {}

PS C:\Users\danie\azure-copy>
```

Bloque 2: Despliegue Docker Compose con Dos Contenedores

En esta segunda iteración, la aplicación se estructura en una arquitectura de múltiples contenedores coordinados por Docker Compose. Esta arquitectura divide el monolito en:

1. **Frontend API (Gateway):** Un contenedor FastAPI en el puerto 80 que recibe todas las peticiones externas y las redirige hacia la capa interna.
2. **Backend API:** El contenedor FastAPI original en el puerto 8080 que se comunica exclusivamente con la API de Gemini, protegido del exterior.

2.1. Estructura de Docker Compose

En la raíz del proyecto se ha introducido un archivo `docker-compose.yml` que orquesta ambos servicios. Este archivo ya cuenta con el nombre y URL de su ACR resueltos directamente en las imágenes, por lo que no requiere sustitución dinámica de variables:

```
version: '3.8'

services:
  backend:
    build: ./backend
    image: planservicioia.azurecr.io/fastapi-backend:v1
    environment:
      - GEMINI_API_KEY=${GEMINI_API_KEY}
    expose:
      - "8080"

  frontend:
    build: ./frontend
    image: planservicioia.azurecr.io/fastapi-frontend:v1
    environment:
      - BACKEND_URL=http://backend:8080
    ports:
      - "80:80"
    depends_on:
      - backend
```

2.2. Pruebas Locales (Docker Compose)

En lugar de construir una sola imagen, utiliza compose para levantar el clúster entero simultáneamente:

⚠ **Prerequisito:** Comprueba que **Docker Desktop** está en ejecución antes de continuar (icono en la barra de tareas → *"Docker Desktop is running"*).

```
# Leer la API Key desde el archivo .env
$env:GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" |
ForEach-Object { $_.ToString().Split("=",2)[1] })
```

```
PS C:\Users\danie\azure-copy> # Leer la API Key desde el archivo .env
>> $env:GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" | ForEach-Object { $_.ToString().Split("=",2)[1] })
PS C:\Users\danie\azure-copy> █
```

```
# Construir y levantar los dos contenedores
docker-compose up --build -d
```

Verifique apuntando al puerto 80 (el cual ahora es gestionado por el Gateway frontend):

```
Invoke-WebRequest "http://localhost:80/generate?prompt=Prueba_Compose"
```

```

PS C:\Users\danie\azure-assignment> docker-compose up --build -d
>
time="2026-06-02T23:39:03+02:00" level=warning msg="C:\Users\danie\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Building 2.1s (28/28) FINISHED
-> [internal] load local bake definitions                                0.0s
-> -> reading from stdin 1.88s                                         0.0s
-> [backend internal] load build definition from Dockerfile            0.0s
-> -> transferring dockerfile: 500B                                     0.0s
-> [frontend internal] load build definition from Dockerfile          0.0s
-> -> transferring dockerfile: 487B                                     0.0s
-> [frontend internal] load metadata for docker.io/library/python:3.11-slim 0.9s
-> [frontend internal] load .dockerignore                             0.0s
-> -> transferring context: 2B                                          0.0s
-> [backend internal] load .dockerignore                             0.0s
-> -> transferring context: 2B                                          0.0s
-> [backend 1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab8b966bc4e91546a833c22893cb848988979487a9fc8e6e38295747e48ac8 0.0s
-> -> resolve docker.io/library/python:3.11-slim@sha256:a3ab8b966bc4e91546a833c22893cb848988979487a9fc8e6e38295747e48ac8 0.0s
-> [frontend internal] load build context                             0.0s
-> -> transferring context: 63B                                         0.0s
-> [backend internal] load build context                             0.0s
-> -> transferring context: 63B                                         0.0s
-> CACHED [backend 2/5] WORKDIR /app                                   0.0s
-> CACHED [frontend 3/5] COPY requirements.txt .                      0.0s
-> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [frontend 5/5] COPY app.py .                                0.0s
-> CACHED [backend 3/5] COPY requirements.txt .                      0.0s
-> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [backend 5/5] COPY app.py .                                0.0s
-> [frontend] exporting to image                                       0.2s
-> -> exporting layers                                                  0.0s
-> -> exporting manifest sha256:bec1f9feca0fc5b6b747af4c28f4c1aceeb5c77d671554618b42b2834a38b857 0.0s
-> -> exporting config sha256:8f396658f32778dc8d8dbcb88b6aa1a7caeb6cb449616898c867dd18b6953128 0.0s
-> -> exporting attestation manifest sha256:bed587f98734aa157b2a2bb2cad862f827e91b3b73435a6a164b8cef9c9f6cb 0.1s
-> -> exporting manifest list sha256:9567ab02470844f4f0438d7d7b7c6a6bc5dc3e72a28ba175cbb5db835a3a148 0.0s
-> -> naming to planservicioia.azurecr.io/fastapi-frontend:v1        0.0s
-> -> unpacking to planservicioia.azurecr.io/fastapi-frontend:v1    0.0s
-> [backend] exporting to image                                       0.2s
-> -> exporting layers                                                  0.0s
-> -> exporting manifest sha256:71a44f5f68d4f28c956f5c308928574661616b078b72c79ba4f8b0f5781c5d 0.0s
-> -> exporting config sha256:bfc579c78f641a871976a82f0f478e4bbd477af63a37fbb358fcb1891927d 0.0s
-> -> exporting attestation manifest sha256:3464b41df02a01bb65326f8ec71d9374f53bfc4a658728c24fcbcaa74cc0d4dd 0.1s
-> -> exporting manifest list sha256:d1523252c6d88c59ea17a53f822e11b282ca88bdc89273a43a53d678a929e4ef 0.0s
-> -> naming to planservicioia.azurecr.io/fastapi-backend:v1        0.0s
-> -> unpacking to planservicioia.azurecr.io/fastapi-backend:v1    0.0s
-> [frontend] resolving provenance for metadata file                 0.0s
-> [backend] resolving provenance for metadata file                 0.0s
[*] Running 2/2
✓ planservicioia.azurecr.io/fastapi-frontend:v1 Built              0.0s
[*] Running 4/4ia.azurecr.io/fastapi-backend:v1 Built            0.0s
✓ planservicioia.azurecr.io/fastapi-frontend:v1 Built            0.0s
✓ planservicioia.azurecr.io/fastapi-backend:v1 Built            0.0s
✓ Container azure-assignment-backend-1 Started                   2.0s
✓ Container azure-assignment-frontend-1 Started                  1.4s
PS C:\Users\danie\azure-assignment> Invoke-WebRequest "http://localhost:88/generate?prompt=Prueba_Compose"

StatusCode      : 200
StatusDescription : OK
Content         : {"prompt":"Prueba_Compose","text":"¡Claro! Parece que quieres una introducción o una \"prueba\" de Jetpack Compose. ¡Es una excelente elección! Jetpack Compose es el kit de herramientas moderno de And...
RawContent      : HTTP/1.1 200 OK
                 Date: Tue, 02 Jun 2026 21:39:18 GMT
                 Server: uvicorn
                 Content-Length: 17023
                 Content-Type: application/json

                 {"prompt":"Prueba_Compose","text":"¡Claro! Parece que quieres una introd...
Headers         : [{"Date", System.String[]}, {"Server", System.String[]}, {"Content-Length", System.String[]}, {"Content-Type", System.String[]}]
Images          : {}
InputFields     : {}
Links           : {}
RawContentLength : 17023
RelationLink    : {}

```

2.3. Despliegue en Azure App Service Multi-Container

Para desplegar la aplicación en Azure usando esta topología, los comandos a ejecutar cambian ligeramente. Si has iniciado una nueva sesión de terminal, redefine primero las variables necesarias:

```

$RESOURCE_GROUP = "gruporecursosia"
$LOCATION          = "spaincentral"
$ACR_NAME         = "planservicioia"
$APP_SERVICE_PLAN = "plan-fastapi-gemini"
$WEB_APP_NAME     = "webapp-fastapi-gemini"
$ACR_PASSWORD     = (az acr credential show --name $ACR_NAME --query
"passwords[0].value" -o tsv)

```

```
PS C:\Users\danie\azure-copy> $RESOURCE_GROUP = "gruporecursosia"
>> $LOCATION = "spaincentral"
>> $ACR_NAME = "planservicioia"
>> $APP_SERVICE_PLAN = "plan-fastapi-gemini"
>> $WEB_APP_NAME = "webapp-fastapi-gemini"
>> $ACR_PASSWORD = (az acr credential show --name $ACR_NAME --query "passwords[0].value" -o tsv)
PS C:\Users\danie\azure-copy>
```

A. Construcción y subida de ambas imágenes al ACR

```
# Exportamos el nombre del ACR para los tags del Compose
$env:ACR_NAME=$ACR_NAME
```

```
# Autenticarse en el ACR antes de hacer push
docker login "$ACR_NAME.azurecr.io" --username $ACR_NAME --password $ACR_PASSWORD
```

```
PS C:\Users\danie\azure-copy> # Obtener la contraseña de administrador del registro
>> $ACR_PASSWORD = (az acr credential show --name $ACR_NAME --query "passwords[0].value" -o tsv)
>> # Iniciar sesión local en Docker de forma segura usando echo y pipe
>> echo $ACR_PASSWORD | docker login "$ACR_NAME.azurecr.io" --username $ACR_NAME --password-stdin
Login Succeeded
PS C:\Users\danie\azure-copy>
```

```
# Construimos y subimos las imágenes
docker-compose build
docker-compose push
```

```
# Crear el App Service Plan (si no lo tienes creado del ejercicio anterior)
az appservice plan create --name $APP_SERVICE_PLAN --resource-group
$RESOURCE_GROUP --sku B1 --is-linux

# Crear la Web App pasándole el docker-compose.yml directamente
az webapp create --resource-group $RESOURCE_GROUP --plan $APP_SERVICE_PLAN --name
$WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file
docker-compose.yml
```

```

PS C:\Users\danie\azure-assignment> az appservice plan create --name $APP_SERVICE_PLAN --resource-group $RESOURCE_GROUP --sku B1 --is-linux
>>
{
  "asyncScalingEnabled": false,
  "elasticScaleEnabled": false,
  "extendedLocation": null,
  "freeOfferExpirationTime": null,
  "geoRegion": "Spain Central",
  "hostingEnvironmentProfile": null,
  "hyperV": false,
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia/providers/Microsoft.Web/serverfarms/plan-fastapi-gemini",
  "isSpot": false,
  "isXenon": false,
  "kind": "linux",
  "kubeEnvironmentProfile": null,
  "location": "spaincentral",
  "maximumElasticWorkerCount": 1,
  "maximumNumberOfWorkers": 3,
  "name": "plan-fastapi-gemini",
  "numberOfSites": 1,
  "numberOfWorkers": 1,
  "perSiteScaling": false,
  "provisioningState": "Succeeded",
  "reserved": true,
  "resourceGroup": "gruporecursosia",
  "sku": {
    "capabilities": null,
    "capacity": 1,
    "family": "B",
    "locations": null,
    "name": "B1",
    "size": "B1",
    "skuCapacity": null,
    "tier": "Basic"
  },
  "spotExpirationTime": null,
  "status": "Ready",
  "subscription": "884c87e5-7027-435d-905b-1722c9bcfddc",
  "tags": null,
  "targetWorkerCount": 0,
  "targetWorkerSizeId": 0,
  "type": "Microsoft.Web/serverfarms",
  "workerTierName": null,
  "zoneRedundant": false
}
PS C:\Users\danie\azure-assignment> ^C

PS C:\Users\danie\azure-assignment> az webapp create --resource-group $RESOURCE_GROUP --plan $APP_SERVICE_PLAN --name $WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file docker-compose.yml
Webapp 'webapp-fastapi-gemini' already exists. The command will use the existing app's settings.
vnet_route_all_enabled is not a known attribute of class <class 'azure.mgmt.web.v2024_11_01.models._models_py3.Site'> and will be ignored
{
  "autoGeneratedDomainNameLabelScope": null,
  "availabilityState": "Normal",
  "clientAffinityEnabled": true,
  "clientAffinityPartitioningEnabled": null,
  "clientAffinityProxyEnabled": false,
  "clientCertEnabled": false,
  "clientCertExclusionPaths": null,
  "clientCertMode": "Required",
  "cloningInfo": null,
  "containerSize": 0,
  "customDomainVerificationId": "1fE448438076E83CD09F2F8CD94F032201C423F2A99F0C84D6FA3298A1AE47",
  "dailyMemoryTimeQuota": 0,
  "daprConfig": null,
  "defaultHostName": "webapp-fastapi-gemini.azurewebsites.net",
  "dnsConfiguration": {
    "dnsAltServer": null,
    "dnsLegacySortOrder": null,
    "dnsMaxCacheTimeout": null,
    "dnsRetryAttemptCount": null,
    "dnsRetryAttemptTimeout": null,
    "dnsServers": null
  },
  "enabled": true,
  "enabledHostNames": [
    "webapp-fastapi-gemini.azurewebsites.net",
    "webapp-fastapi-gemini.scm.azurewebsites.net"
  ],
  "endToEndEncryptionEnabled": false,
  "extendedLocation": null,
  "ftpPublishingUrl": "https://waws-prod-esc-003.ftp.azurewebsites.windows.net/site/wwwroot",
  "functionAppConfig": null,
  "hostNameSslStates": [
    {
      "certificateResourceId": null,
      "hostType": "Standard",
      "ipBasedSslResult": null,
      "ipBasedSslState": "NotConfigured",
      "name": "webapp-fastapi-gemini.azurewebsites.net",
      "sslState": "Disabled",
      "thumbprint": null,
      "toUpdate": null,
      "toUpdateIpBasedSsl": null,
      "virtualIPv6": null,
      "virtualIp": null
    },
    {
      "certificateResourceId": null,
      "hostType": "Repository",
      "ipBasedSslResult": null,
      "ipBasedSslState": "NotConfigured",
      "name": "webapp-fastapi-gemini.scm.azurewebsites.net",
      "sslState": "Disabled",
      "thumbprint": null,
      "toUpdate": null,
      "toUpdateIpBasedSsl": null,
      "virtualIPv6": null,
      "virtualIp": null
    }
  ]
}

```

C. Configuración de Variables en Azure

```
# Vincular las credenciales del ACR a la Web App (usando parámetros actualizados)
az webapp config container set --name $WEB_APP_NAME --resource-group
$RESOURCE_GROUP --container-registry-url "https://$ACR_NAME.azurecr.io" --
container-registry-user $ACR_NAME --container-registry-password $ACR_PASSWORD

# Leer la API Key desde el archivo .env
$GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" | ForEach-
Object { $_.ToString().Split("=",2)[1] })

# Definir la API Key de Gemini
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --settings GEMINI_API_KEY=$GEMINI_API_KEY

# Configurar el puerto público: Indicamos a Azure que envíe el tráfico al puerto
80 (Frontend)
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --settings WEBSITES_PORT=80

# Configurar comunicación interna: En Azure Multi-container los contenedores se
comunican por localhost
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --settings BACKEND_URL="http://localhost:8080"

# Iniciar la Web App (asegura el arranque si la aplicación está detenida)
az webapp start --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

```
PS C:\Users\danie\azure-copy> # Configurar el puerto público: Indicamos a Azure que envíe el tráfico al puerto 80 (Frontend)
>> az webapp config appsettings set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --settings WEBSITES_PORT=80
>>
>> # Configurar comunicación interna: En Azure Multi-container los contenedores se comunican por localhost
>> az webapp config appsettings set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --settings BACKEND_URL="http://localhost:8080"
>>
>> # Iniciar la Web App (asegura el arranque si la aplicación está detenida)
>> az webapp start --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
[
  {
    "name": "WEBSITES_ENABLE_APP_SERVICE_STORAGE",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITES_PORT",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "GEMINI_API_KEY",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_URL",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_USERNAME",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_PASSWORD",
    "slotSetting": false,
    "value": null
  }
]
[
  {
    "name": "WEBSITES_ENABLE_APP_SERVICE_STORAGE",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITES_PORT",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "GEMINI_API_KEY",
    "slotSetting": false,
    "value": null
  }
]
```

```

    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_URL",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_USERNAME",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_PASSWORD",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "BACKEND_URL",
    "slotSetting": false,
    "value": null
  }
]
PS C:\Users\danie\azure-copy>

```

D. Prueba del Despliegue en Azure Una vez que App Service haya descargado las imágenes y levantado los contenedores (esto puede tomar 2-3 minutos la primera vez), puedes probar que el flujo multi-contenedor funciona haciendo una llamada a tu dominio `.azurewebsites.net`:

```

# Realizar petición de prueba a la API Frontend desplegada
Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?
prompt=Test_Compose_Azure" -UseBasicParsing

```

```

PS C:\Users\danie\azure-copy> Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?prompt=Test_Compose_Azure" -UseBasicParsing

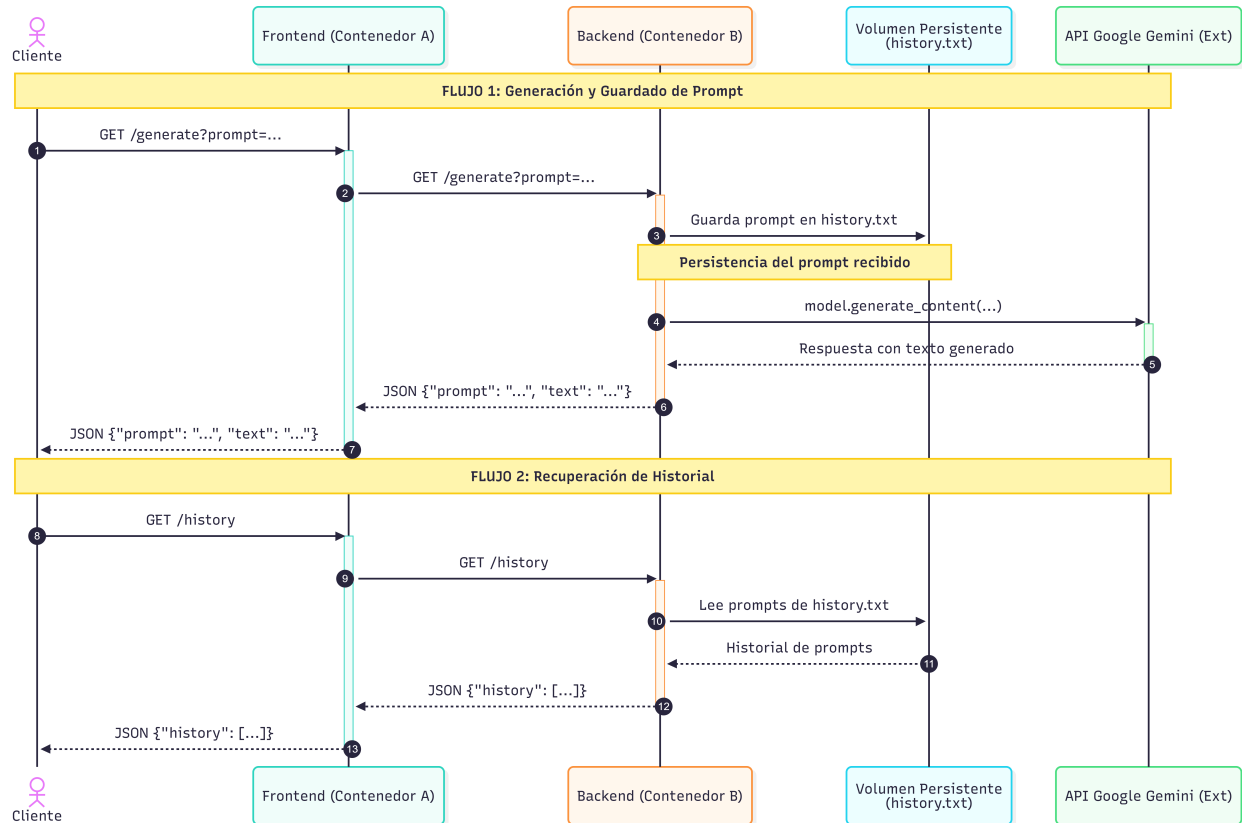
StatusCode      : 200
StatusDescription : OK
Content          : {"prompt":"Test_Compose_Azure","text":"The phrase \"Test_Compose_Azure\" suggests a few interconnected concepts related to developing, testing, and deploying containerized applications, often using Do...
RawContent       : HTTP/1.1 200 OK
                   Date: Mon, 29 Jun 2026 00:27:43 GMT
                   Server: uvicorn
                   Set-Cookie: ARRAffinity=c8fcc30ceb9d725a7f379f8837c91f9e4afa35242bd4fb5dcad86baef02d41ad;Path=/;HttpOnly;Secure;Domain=webapp-fas...
Headers          : {[Date, System.String[]], [Server, System.String[]], [Set-Cookie, System.String[]], [Content-Length, System.String[]]...}
Images           : {}
InputFields      : {}
Links            : {}
RawContentLength : 14421
RelationLink     : {}
PS C:\Users\danie\azure-copy>

```

(Nota: Si obtienes un error inicial o un tiempo de espera agotado, espera un par de minutos a que los contenedores terminen de arrancar o ejecuta `az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP` para forzar un reinicio).

(Nota para actualizaciones: Si realizas modificaciones en el archivo `docker-compose.yml` localmente y deseas subirlas a Azure, debes volver a cargar la configuración en la Web App ejecutando: `az webapp config container set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file docker-compose.yml`).

Bloque 3: Despliegue Compose con Persistencia



Para asegurar que las consultas realizadas a la aplicación no se pierdan cuando se reinician los contenedores o se vuelve a desplegar el servicio en la nube, se ha implementado un mecanismo de almacenamiento persistente montando un recurso compartido de archivos de Azure (Azure File Share) directamente en los contenedores.

El servicio Backend guarda cada prompt en formato de texto plano dentro de la ruta `/app/data/history.txt`. Esta ruta está mapeada a un almacenamiento externo y permanente.

3.1. Aprovisionamiento de Azure Files y Conexión con la Web App

Sigue estos pasos en la terminal PowerShell para crear el almacenamiento persistente y conectarlo a tu Web App:

Paso 1: Definir variables de entorno adicionales

```
# Variables del entorno de Azure (redefinir si es una sesión de terminal nueva)
$RESOURCE_GROUP = "gruporecursosia"
$LOCATION         = "spaincentral"
$WEB_APP_NAME   = "webapp-fastapi-gemini"

# Variables para el almacenamiento persistente
$STORAGE_ACCOUNT = "almacenamientoappia" # Debe ser un nombre único globalmente
$SHARE_NAME      = "archivos-texto"
$MOUNT_PATH      = "/app/data" # Ruta dentro del contenedor donde se guardará el historial
```

```
PS C:\Users\danie\azure-copy> # Variables del entorno de Azure (redefinir si es una sesión de terminal nueva)
>> $RESOURCE_GROUP = "gruporecursosia"
>> $LOCATION = "spaincentral"
>> $WEB_APP_NAME = "webapp-fastapi-gemini"
>>
>> # Variables para el almacenamiento persistente
>> $STORAGE_ACCOUNT = "almacenamientoappia" # Debe ser un nombre único globalmente
>> $SHARE_NAME = "archivos-texto"
>> $MOUNT_PATH = "/app/data" # Ruta dentro del contenedor donde se guardará el historial
PS C:\Users\danie\azure-copy>
```

Paso 2: Crear la Infraestructura de Almacenamiento

```
# 1. Crear la Cuenta de Almacenamiento (Storage Account)
az storage account create --name $STORAGE_ACCOUNT --resource-group $RESOURCE_GROUP
--location $LOCATION --sku Standard_LRS

# 2. Crear el recurso compartido de archivos (File Share)
az storage share-rm create --resource-group $RESOURCE_GROUP --storage-account
$STORAGE_ACCOUNT --name $SHARE_NAME --quota 5
```

```
PS C:\Users\danie\azure-copy> # 1. Crear la Cuenta de Almacenamiento (Storage Account)
>> az storage account create --name $STORAGE_ACCOUNT --resource-group $RESOURCE_GROUP --location $LOCATION --sku Standard_LRS
>>
>> # 2. Crear el recurso compartido de archivos (File Share)
>> az storage share-rm create --resource-group $RESOURCE_GROUP --storage-account $STORAGE_ACCOUNT --name $SHARE_NAME --quota 5
A storage account with the provided name almacenamientoappia is found. Will continue to update the existing account.
{
  "accessTier": "Hot",
  "accountMigrationInProgress": null,
  "allowBlobPublicAccess": false,
  "allowCrossTenantReplication": false,
  "allowSharedKeyAccess": null,
  "allowedCopyScope": null,
  "azureFilesIdentityBasedAuthentication": null,
  "blobRestoreStatus": null,
  "creationTime": "2026-06-03T18:39:53.057487+00:00",
  "customDomain": null,
  "defaultToOAuthAuthentication": null,
  "dnsEndpointType": null,
  "dualStackEndpointPreference": null,
  "enableExtendedGroups": null,
  "enableHttpsTrafficOnly": true,
  "enableNfsV3": null,
  "encryption": {
    "encryptionIdentity": null,
    "keySource": "Microsoft.Storage",
    "keyVaultProperties": null,
    "requireInfrastructureEncryption": null,
    "services": {
      "blob": {
        "enabled": true,
```

Paso 3: Conectar el Almacenamiento a la Web App

```
# 1. Obtener la clave de acceso del Storage Account
$STORAGE_KEY = (az storage account keys list --resource-group $RESOURCE_GROUP --
account-name $STORAGE_ACCOUNT --query "[0].value" --output tsv)
```

```
PS C:\Users\danie\azure-copy> $STORAGE_KEY = (az storage account keys list --resource-group $RESOURCE_GROUP --account-name $STORAGE_ACCOUNT --query "[0].value" --output tsv)
PS C:\Users\danie\azure-copy>
```

```
# 2. (Opcional) Eliminar un montaje previo con el mismo ID si existe
az webapp config storage-account delete --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --custom-id AlmacenamientoPersistente

# 3. (Opcional) Limpiar el historial previo en la nube para empezar con una demo
limpia (si no existe el archivo, dará error, el cual se puede ignorar)
az storage file delete --share-name $SHARE_NAME --account-name $STORAGE_ACCOUNT --
account-key $STORAGE_KEY --path history.txt

# 4. Montar el almacenamiento persistente en la Web App
az webapp config storage-account add --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --custom-id AlmacenamientoPersistente --storage-type AzureFiles --
account-name $STORAGE_ACCOUNT --share-name $SHARE_NAME --access-key $STORAGE_KEY -
-mount-path $MOUNT_PATH

# 4. Reiniciar la Web App para que el montaje surta efecto
az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

```
PS C:\Users\danie\azure-assignment> az storage share-rm create --resource-group $RESOURCE_GROUP --storage-account $STORAGE_ACCOUNT --name $SHARE_NAME --quota 5
{
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfdcc/resourceGroups/gruporecursosia/providers/Microsoft.Storage/storageAccounts/almacenamientoappia/fileServices/default/shares/archivos-texto",
  "name": "archivos-texto",
  "resourceGroup": "gruporecursosia",
  "shareQuota": 5,
  "type": "Microsoft.Storage/storageAccounts/fileServices/shares"
}
PS C:\Users\danie\azure-assignment> az webapp config storage-account delete --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --custom-id AlmacenamientoPersistente
Storage account access keys have been redacted. Use 'az webapp config storage-account list' to view.
{}
PS C:\Users\danie\azure-assignment> az webapp config storage-account add --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --custom-id AlmacenamientoPersistente --storage-type AzureFiles --account-name $STORAGE_ACCOUNT --share-na
me $SHARE_NAME --access-key $STORAGE_KEY --mount-path $MOUNT_PATH
Storage account access keys have been redacted. Use 'az webapp config storage-account list' to view.
{
  "AlmacenamientoPersistente": {
    "accessKey": null,
    "accountName": "almacenamientoappia",
    "mountPath": "/app/data",
    "protocol": "Smb",
    "shareName": "archivos-texto",
    "state": "Ok",
    "type": "AzureFiles"
  }
}
PS C:\Users\danie\azure-assignment>
```

3.2. Ajuste al Docker Compose (`docker-compose.yml`)

Para soportar las pruebas y persistencia en local, se actualiza el `docker-compose.yml` añadiendo el volumen persistente al servicio `backend` y etiquetando las imágenes como `v2`. El archivo completo queda así:

```
version: '3.8'

services:
  backend:
    build: ./backend
    image: planservicioia.azurecr.io/fastapi-backend:v2
    environment:
      - GEMINI_API_KEY=${GEMINI_API_KEY}
    expose:
      - "8080"
    volumes:
      - AlmacenamientoPersistente:/app/data

  frontend:
```

```
build: ./frontend
image: planservicioia.azurecr.io/fastapi-frontend:v2
environment:
  - BACKEND_URL=http://backend:8080
ports:
  - "80:80"
depends_on:
  - backend

volumes:
  AlmacenamientoPersistente:
```

- **En Local:** docker-compose crea y gestiona automáticamente un volumen local llamado `AlmacenamientoPersistente` que se monta en `/app/data` y persiste los datos de tus consultas.
- **En Azure:** Azure App Service intercepta el nombre del volumen (`AlmacenamientoPersistente`) y lo asocia automáticamente con el montaje de Azure Files que tiene el mismo `custom-id` (Paso 3), guardando el archivo de texto directamente en la cuenta de almacenamiento en la nube sin depender del almacenamiento efímero del contenedor.

3.3. Pruebas de Persistencia en Local

⚠ **Prerequisito:** Comprueba que **Docker Desktop** está en ejecución antes de continuar (icono en la barra de tareas → *"Docker Desktop is running"*).

1. Levanta los contenedores en local:

```
docker-compose up --build -d
```

2. Envía prompts de prueba al endpoint público de generación (puerto 80):

```
Invoke-WebRequest "http://localhost:80/generate?prompt=Local_Prompt_1" -
UseBasicParsing
Invoke-WebRequest "http://localhost:80/generate?prompt=Local_Prompt_2" -
UseBasicParsing
```

3. Verifica que se ha creado el historial accediendo al endpoint `/history`:

```
(Invoke-WebRequest "http://localhost:80/history" -UseBasicParsing).Content
# Salida esperada: {"history":["Local_Prompt_1"]}
```

```

PS C:\Users\daniel\azure-assignment> docker-compose up --build -d
time="2026-06-03T21:28:42.00" level=warning msg="C:\Users\daniel\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Building 2.3s (cache) #24M24P
--> [internal] load local bake definitions 0.0s
--> reading from stdin 1.000s 0.0s
--> [backend internal] load build definition from Dockerfile 0.0s
--> --> transferring Dockerfile: 500s 0.0s
--> [frontend internal] load build definition from Dockerfile 0.0s
--> --> transferring Dockerfile: 407s 0.0s
--> [backend internal] load metadata for docker.io/library/python:3.11-slim 1.1s
--> [frontend internal] load .dockerignore 0.0s
--> --> transferring context: 28 0.0s
--> [backend internal] load .dockerignore 0.0s
--> --> transferring context: 28 0.0s
--> [backend 1/5] FROM docker.io/library/python:3.11-slim@sha256:abab8966dc4e1546a813c22893c38498807 0.0s
--> --> resolve docker.io/library/python:3.11-slim@sha256:abab8966dc4e1546a813c22893c384988079487a9fc 0.0s
--> [frontend internal] load build context 0.0s
--> --> transferring context: 838 0.0s
--> [backend internal] load build context 0.0s
--> --> transferring context: 638 0.0s
--> CACHED [frontend 2/5] WORKDIR /app 0.0s
--> CACHED [backend 3/5] COPY requirements.txt . 0.0s
--> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
--> CACHED [backend 5/5] COPY app.py . 0.0s
--> CACHED [frontend 3/5] COPY requirements.txt . 0.0s
--> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
--> CACHED [frontend 5/5] COPY app.py . 0.0s
--> [frontend] exporting to image 0.2s
--> --> exporting layers 0.0s
--> --> exporting manifest sha256:9b3d418173151fcd0f8a338b1f1bdf1d3b125c3d884f9c2d77f94c0fff 0.0s
--> --> exporting config sha256:b6f1eb334806876d73428edca72676f61acfb27b672764f92721a382433c 0.0s --> exporting config sha256:6f72b6676edf93cc421ab2c0d7a28fdd82a7d4e4cd818058f1 0.1s
--> --> exporting attestation manifest sha256:42e748d634b6151d98acc34c76f9cc11919e9b0b0a3c74d1bdcf5d 0.1s
--> --> exporting manifest list sha256:1569880963f57a34fccc808048994221c61109f46a88880800196 0.0s
--> --> naming to plaveserviciola.azurecr.io/fastapi-frontend:v2 0.0s
--> --> unpacking to plaveserviciola.azurecr.io/fastapi-frontend:v2 0.0s
--> [frontend] finalizing previous for metadata file 0.0s
--> [backend] finalizing previous for metadata file 0.0s
[+] Running 6/6
✔ plaveserviciola.azurecr.io/fastapi-backend:v2 Built 0.0s
✔ plaveserviciola.azurecr.io/fastapi-frontend:v2 Built 0.0s
✔ Network azure-assignment_default Created 0.1s
✔ Volume azure-assignment_almacenasintoPersistente Created 0.0s
✔ Container azure-assignment-backend-1 Started 0.0s
✔ Container azure-assignment-frontend-1 Started 0.0s
PS C:\Users\daniel\azure-assignment> Invoke-WebRequest "http://localhost:80/generate/prompt-local_prompt_1"

StatusCode : 200
StatusDescription : OK
Content : {"prompt":"Local_Prompt_1","text":"\"Local_Prompt_1\" seems to be a placeholder or a general reference.\\n\\nlo help you with it, I need more information. Could you please tell me what. \"What is the ...\""}
RawContent : HTTP/1.1 200 OK
Content-Length : 655
Content-Type : application/json
Date: Wed, 03 Jun 2026 21:28:37 GMT
Server: uvicorn

{"prompt":"Local_Prompt_1","text":"\"Local_Prompt_1\" seems to be a plac..."}
Forms : {}
Headers : [{"Content-Length": 655}, {"Content-Type": "application/json"}, {"Date": "Wed, 03 Jun 2026 21:28:37 GMT"}, {"Server": "uvicorn"}]
Images : {}
InputFields : {}
Links : {}

PS C:\Users\daniel\azure-assignment> (Invoke-WebRequest "http://localhost:80/history").Content ("history":["Local_Prompt_1"]) PS C:\Users\daniel\azure-assignment> docker-compose down
time="2026-06-03T21:28:53:02:00" level=warning msg="C:\Users\daniel\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
✔ Container azure-assignment-frontend-1 Removed 0.5s
✔ Container azure-assignment-backend-1 Removed 0.7s
✔ Network azure-assignment_default Removed 0.4s
PS C:\Users\daniel\azure-assignment> docker-compose up -d
time="2026-06-03T21:21:01:02:00" level=warning msg="C:\Users\daniel\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
✔ Network azure-assignment_default Created 0.1s
✔ Container azure-assignment-backend-1 Started 0.0s
✔ Container azure-assignment-frontend-1 Started 0.0s
PS C:\Users\daniel\azure-assignment> (Invoke-WebRequest "http://localhost:80/history").Content
{"history":["Local_Prompt_1"]}
PS C:\Users\daniel\azure-assignment>

```

4. Detén y elimina por completo los contenedores:

```
docker-compose down
```

5. Vuelve a levantar el servicio:

```
docker-compose up -d
```

6. Consulta nuevamente el historial.

```
(Invoke-WebRequest "http://localhost:80/history" -UseBasicParsing).Content
# Salida esperada: {"history":["Local_Prompt_1"]}
```

Los datos deben seguir presentes puesto que están guardados en el archivo `./data/history.txt` de tu equipo local y sobreviven a la recreación de los contenedores.

```

PS C:\Users\danie\azure-copy> docker-compose down
>>
time="2026-06-29T15:54:31+02:00" level=warning msg="C:\\Users\\danie\\azure-copy\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
id potential confusion"
[+] Running 3/3
  ✓ Container azure-copy-frontend-1 Removed                                0.6s
  ✓ Container azure-copy-backend-1   Removed                                0.8s
  ✓ Network azure-copy_default       Removed                                0.4s
PS C:\Users\danie\azure-copy> docker-compose up -d
>>
time="2026-06-29T15:54:41+02:00" level=warning msg="C:\\Users\\danie\\azure-copy\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
id potential confusion"
[+] Running 3/3
  ✓ Network azure-copy_default       Created                                0.1s
  ✓ Container azure-copy-backend-1   Started                                0.5s
  ✓ Container azure-copy-frontend-1  Started                                0.7s
PS C:\Users\danie\azure-copy> (Invoke-WebRequest "http://localhost:80/history" -UseBasicParsing).Content
>> # Salida esperada: {"history":["Local_Prompt_1"]}
{"history":["Local_Prompt_1"]}
PS C:\Users\danie\azure-copy>

```

3.4. Pruebas de Persistencia en Azure

1. Reconstruye y sube las imágenes al ACR con los nuevos cambios de código:

```

$env:ACR_NAME=$ACR_NAME
docker-compose build
docker-compose push

```

```

PS C:\Users\danie\azure-copy> $env:ACR_NAME=$ACR_NAME
>> docker-compose build
>> docker-compose push
time="2026-06-29T16:44:56+02:00" level=warning msg="C:\\Users\\danie\\azure-copy\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Building 2.2s (20/20) FINISHED
  -> [internal] load local bake definitions                                0.0s
  -> reading from stdin 1.85s                                           0.0s
  -> [backend internal] load build definition from Dockerfile             0.0s
  -> transferring dockerfile: 508B                                       0.0s
  -> [frontend internal] load build definition from Dockerfile           0.0s
  -> transferring dockerfile: 507B                                       0.0s
  -> [frontend internal] load metadata for docker.io/library/python:3.11-slim 1.2s
  -> [frontend internal] load .dockerignore                               0.0s
  -> transferring context: 2B                                             0.0s
  -> [backend internal] load .dockerignore                                 0.0s
  -> transferring context: 2B                                             0.0s
  -> [backend 1/5] FROM docker.io/library/python:3.11-slim@sha256:b27df5841f3355e9473f9a516d38a6783b6c8dfcaaf2d14a240f443b368ddb6 0.0s
  -> resolve docker.io/library/python:3.11-slim@sha256:b27df5841f3355e9473f9a516d38a6783b6c8dfcaaf2d14a240f443b368ddb6 0.0s
  -> [backend internal] load build context                                0.0s
  -> transferring context: 1.70kB                                         0.0s
  -> [frontend internal] load build context                                0.0s
  -> transferring context: 1.42kB                                         0.0s
  -> CACHED [backend 2/5] WORKDIR /app                                    0.0s
  -> CACHED [frontend 3/5] COPY requirements.txt .                        0.0s
  -> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
  -> CACHED [frontend 5/5] COPY app.py .                                  0.0s
  -> CACHED [backend 3/5] COPY requirements.txt .                        0.0s
  -> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
  -> CACHED [backend 5/5] COPY app.py .                                  0.0s
  -> [backend] exporting to image                                         0.2s
  -> exporting layers                                                    0.0s
  -> exporting manifest sha256:d418aefbc30ff35f1d024c38a209b59a8b67047cd36be6e91b32a8ed5c6c7ef 0.0s
  -> exporting config sha256:84331d3c3c55e9a1896d72b3e086942072384c28d1d4242790f845632c91b55 0.0s
  -> exporting attestation manifest sha256:86be4d5e4e45205680d5f47c7e9f98c1dc86813af0a2e41cbb01261c868920796 0.1s
  -> exporting manifest list sha256:52ff7b452808ec83dbd75840a2be368ba6384213237bf3c2a24d7b4668499178 0.0s
  -> naming to planservicioia.azurecr.io/fastapi-backend:v2            0.0s
  -> unpacking to planservicioia.azurecr.io/fastapi-backend:v2          0.0s
  -> [frontend] exporting to image                                         0.2s
  -> exporting layers                                                    0.0s
  -> exporting manifest sha256:611c0084930eac631a2568ac2d3faacd50e569a70ad327cd98e8c1d8c3ba04ca 0.0s
  -> exporting config sha256:9e154fcd348804f7a9924b834b56fa0acbb3da4766e48b07fd20ac9448a6d4c 0.0s
  -> exporting attestation manifest sha256:d678393a8015d57763c75d77fd553dfaa287f8c210d993fbc4d8cf9d9332f3a 0.1s
  -> exporting manifest list sha256:71745ae71dd335bef4ca73951a10c42d8c0fd9409f4c8aaf5c227602b320857c 0.0s
  -> unpacking to planservicioia.azurecr.io/fastapi-frontend:v2         0.0s
  -> [backend] resolving provenance for metadata file                    0.0s
  -> [frontend] resolving provenance for metadata file                    0.0s
[+] Building 2/2
  ✓ planservicioia.azurecr.io/fastapi-backend:v2 Built                  0.0s
  ✓ planservicioia.azurecr.io/fastapi-frontend:v2 Built                  0.0s
time="2026-06-29T16:44:59+02:00" level=warning msg="C:\\Users\\danie\\azure-copy\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Pushing 16/18
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: 733dc49a35fd Layer already exists 0.3s
[+] Pushing 18/18
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: e95a6c7ea704 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: 733dc49a35fd Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: e95a6c7ea704 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: d31bca4e7070 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: ef10b7552742 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: 862dfdf14e40 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: af4d97f637ec Pushed 0.5s
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: d9f0e75d2aea Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: 2bd1111909f3 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v2: e2c1836a1da9 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-backend:v2: e83d1c61dfc6 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-backend:v2: 733dc49a35fd Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-backend:v2: 08235e5914a2 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-backend:v2: 2bd1111909f3 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-backend:v2: e95a6c7ea704 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-backend:v2: 950885aeb182 Pushed 0.5s
  ✓ Pushing planservicioia.azurecr.io/fastapi-backend:v2: d31bca4e7070 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-backend:v2: ef10b7552742 Layer already exists 0.3s
  ✓ Pushing planservicioia.azurecr.io/fastapi-backend:v2: dc95b488ecf Layer already exists 0.3s
PS C:\Users\danie\azure-copy>

```

i Nota: La advertencia (*warning*) relacionada con la versión de Docker Compose en la construcción de imágenes se corregirá más adelante.

2. Actualiza la configuración de la Web App con el nuevo `docker-compose.yml` (que incluye el volumen) y fuerza el reinicio:

```
# Subir la nueva configuración del compose a Azure (incluye el volumen en backend)

az webapp config container set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file docker-compose.yml

# Reiniciar para descargar las nuevas imágenes y aplicar el montaje

az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

```
PS C:\Users\danie\azure-copy> # Subir la nueva configuración del compose a Azure (incluye el volumen en backend)
>> az webapp config container set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file docker-compose.yml
>> # Reiniciar para descargar las nuevas imágenes y aplicar el montaje
>> az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP

{
  {
    "name": "WEBSITES_ENABLE_APP_SERVICE_STORAGE",
    "slotSetting": false,
    "value": "false"
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_URL",
    "slotSetting": false,
    "value": "https://planservicioia.azurecr.io"
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_USERNAME",
    "slotSetting": false,
    "value": "planservicioia"
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_PASSWORD",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_CUSTOM_IMAGE_NAME",
    "value": "COMPOSE[dmyc2lvbjogJzW0CcnCg8Kc2VydmIjZXN6DQogIGYhY2t1bmQ0DQoICAgVnFpbG90GICAgYmFjaZvUzA0KICAgIGlTYWd1O1BwbGFCuc2VydmljaW9pYS5henVhZWYwLmVlZzhc3RhcgkYmFjaZvUzZDp2Ng0KICAgIGVudmAGLSBHRRUjT1k1QVB8X0RFTU0SV9BUe1fSvZfQ0KICAgIGV4c69zZit0NCiAgICAgIC8gIjpwODA0DQoICAgdms9sdWllcz0NCiAgICAgIC8gOktYmVlbnFkaWVudG90ZXJzaXB0ZWN5O2ToYXBlL2RhdcENCg0KICBmc9udGVuZDoNCiAgICVuzA0KICAgIGlTYWd1O1BwbGFCuc2VydmljaW9pYS5henVhZWYwLmVlZzhc3RhcgkYmVubnRlbmQ0dG90KICAgICB1bnZpc9ubWVudDoNCiAgICAgIC8gOkFSOVRFR9Vukw9Ra0KicDovL2JhY2t1bmQ0DAAM48KICAgIHbcnRz0g0KICAgICAgLSA0OD5kc19vbjoNCiAgICAgIC8gYmFjaZvUzA0KDQp2b2x1bmVzOg0KICBB6G1hy2VuYyIpZW50b1BlcnNpc3R1bnR1Og0KDQo="]
  }
}

PS C:\Users\danie\azure-copy>
```

- ### 3. Envía una consulta a la Web App en la nube:

```
Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/generate?prompt=Azure_Prompt_1" -UseBasicParsing
```

4. Consulta el endpoint de historial para comprobar que ha registrado la consulta:

```
(Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/history"
-UseBasicParsing).Content
# Salida esperada: {"history":["Azure Prompt 1"]}
```

```

PS C:\Users\danie\azure-copy> Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/generate?prompt=Azure_Prompt_1" -UseBasicParsing
StatusCode      : 200
StatusDescription : OK
Content         : {"prompt":"Azure_Prompt_1","text":"It seems you're asking for information or a guide related to \"prompting\" specifically within the Azure ecosystem, likely pertaining to Azure AI services like Azure..."}
RawContent      : HTTP/1.1 200 OK
                  Date: Mon, 29 Jun 2026 14:50:35 GMT
                  Server: uvicorn
                  Set-Cookie: ARRAffinity=c8fcc30ceb9d725a7f379f8837c91f9e4afa35242bd4fb5dcad86baef02d41ad;Path=/;HttpOnly;Secure;Domain=webapp-fas...
Headers         : {[Date, System.String[]], [Server, System.String[]], [Set-Cookie, System.String[]], [Content-Length, System.String[]]-}
Images          : {}
InputFields     : {}
Links           : {}
RawContentLength : 11014
RelationLink    : {}

PS C:\Users\danie\azure-copy> (Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/history" -UseBasicParsing).Content
>> # Salida esperada: {"history":["Azure_Prompt_1"]}
{"history":["Azure_Prompt_1"]}
PS C:\Users\danie\azure-copy>

```

5. (Opcional) Verifica la persistencia listando los archivos en Azure File Share directamente mediante Azure CLI:

```

# Listar archivos en el File Share
az storage file list --share-name $SHARE_NAME --account-name
$STORAGE_ACCOUNT --account-key $STORAGE_KEY --output table

# Descargar y mostrar el contenido de history.txt
az storage file download --share-name $SHARE_NAME --account-name
$STORAGE_ACCOUNT --account-key $STORAGE_KEY --path history.txt --dest .
Get-Content history.txt
Remove-Item history.txt

```

```

PS C:\Users\danie\azure-copy> # Listar archivos en el File Share
>> az storage file list --share-name $SHARE_NAME --account-name $STORAGE_ACCOUNT --account-key $STORAGE_KEY --output table
>>
>> # Descargar y mostrar el contenido de history.txt
>> az storage file download --share-name $SHARE_NAME --account-name $STORAGE_ACCOUNT --account-key $STORAGE_KEY --path history.txt --dest .
>> Get-Content history.txt
>> Remove-Item history.txt
Name          Content Length Type      Last Modified
-----
history.txt    15          file
Finished[#####] 100.0000%
{
  "changeTime": "2026-06-29T14:50:35.514239",
  "content": "",
  "creationTime": "2026-06-29T14:50:35.508265",
  "fileAttributes": "Archive",
  "fileId": "9223398425133842432",
  "fileMode": null,
  "fileType": "File",
  "group": null,
  "isDirectory": false,
  "lastAccessTime": null,
  "lastWriteTime": "2026-06-29T14:50:35.514239",
  "lease": {
    "duration": null,
    "state": "available",
    "status": "unlocked"
  },
  "linkCount": null,
  "metadata": {},
  "name": "history.txt",
  "nfsFileType": null,
  "owner": null,
  "parentId": "0",
  "path": "history.txt",
  "permissionKey": "7123878075408946997*14591083470711979963",
  "properties": {
    "contentLength": 15,
    "contentRange": null,
    "contentSettings": {
      "cacheControl": null,
      "contentDisposition": null,
      "contentEncoding": null,
      "contentLanguage": null,
      "contentMd5": null,
      "contentType": "application/octet-stream"
    },
    "copy": {
      "completionTime": null,
      "destinationSnapshot": null,
      "id": null,
      "incrementalCopy": null,
      "progress": null,
      "source": null,
      "status": null,
      "statusDescription": null
    },
    "etag": "\"0x80ED5EDC790C379\"",
    "lastModified": "2026-06-29T14:50:36+00:00",
    "serverEncrypted": true
  },
  "share": "archivos-texto",
  "size": 15,
  "snapshot": null
}
Azure_Prompt_1
PS C:\Users\danie\azure-copy>

```

6. Forza un reinicio de la Web App en Azure para simular un fallo o actualización del servicio:

```
az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

7. Vuelve a realizar la consulta al historial tras esperar un par de minutos:

```
(Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/history"
-UseBasicParsing).Content
```

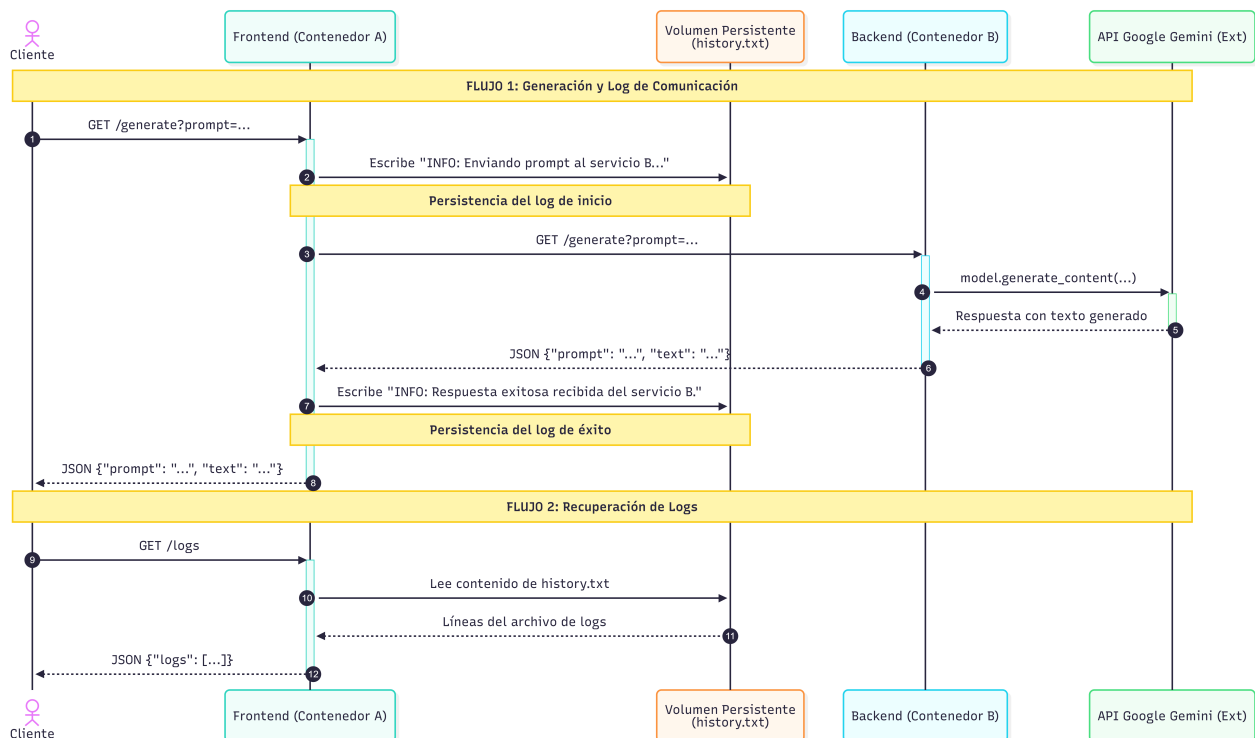
Resultado: El endpoint debe seguir devolviendo `{"history":["Azure_Prompt_1"]}` demostrando que el archivo se almacena en el volumen persistente de Azure Files y no se pierde tras reiniciar el servicio.

```
PS C:\Users\danie\azure-copy> az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
PS C:\Users\danie\azure-copy> (Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/history" -UseBasicParsing).Content
{"history":["Azure_Prompt_1"]}
PS C:\Users\danie\azure-copy>
```

(Nota: Es posible que aparezcan más entradas en la lista del historial si ya habías lanzado peticiones de prueba anteriormente).

Bloque 4: Despliegue Final con Persistencia y Prueba de Interconexión

4.1. Diagrama de Arquitectura Final



En esta nueva iteración de la arquitectura (imágenes etiquetadas con la versión **v3**), se ha modificado la demostración de la comunicación de red entre contenedores de la siguiente forma:

- Reemplazo de `/history`:** Se eliminó la anterior funcionalidad de historial de prompts que gestionaba el Backend en `history.txt`.
- Generación de Logs en Frontend:** Ahora, el Frontend (Contenedor A) usa la librería estándar `logging` de Python para registrar cada comunicación con el Backend (Contenedor B) en el archivo `history.txt`.
- Endpoint `/logs`:** Se habilitó un nuevo endpoint en el Frontend (`GET /logs`) para leer de forma local el archivo de logs `history.txt` y retornar los registros del flujo de peticiones.

4. **Reubicación de Volumen:** El volumen persistente `AlmacenamientoPersistente` ha sido reubicado en el Frontend para conservar el historial de logs ante reinicios.

4.2. Configuración en docker-compose (Versión v3)

El archivo `docker-compose.yml` actualiza las imágenes a la versión `v3` y traslada el volumen persistente al servicio `frontend`:

```
services:
  backend:
    build: ./backend
    image: planservicioia.azurecr.io/fastapi-backend:v3
    environment:
      - GEMINI_API_KEY=${GEMINI_API_KEY}
    expose:
      - "8080"

  frontend:
    build: ./frontend
    image: planservicioia.azurecr.io/fastapi-frontend:v3
    environment:
      - BACKEND_URL=http://backend:8080
    ports:
      - "80:80"
    depends_on:
      - backend
    volumes:
      - AlmacenamientoPersistente:/app/data

volumes:
  AlmacenamientoPersistente:
```

4.3. Definición de los Contenedores (Dockerfiles)

Para la construcción de los contenedores de Frontend y Backend de esta arquitectura final, se utilizan los siguientes archivos `Dockerfile`:

Backend (`backend/Dockerfile`)

```
# Use official Python slim image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements.txt first to leverage Docker cache
COPY requirements.txt .

# Install all dependencies from requirements.txt
```

```
RUN pip install --no-cache-dir -r requirements.txt

# Copy the FastAPI app
COPY app.py .

# Expose FastAPI port
EXPOSE 8080

# Run the app with Uvicorn
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Frontend (frontend/Dockerfile)

```
# Use official Python slim image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements.txt first to leverage Docker cache
COPY requirements.txt .

# Install all dependencies from requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copy the FastAPI app
COPY app.py .

# Expose FastAPI port
EXPOSE 80

# Run the app with Uvicorn on port 80
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "80"]
```

4.4. Pruebas de Logs y Persistencia en Local

 **Prerequisito:** Comprueba que **Docker Desktop** está en ejecución antes de continuar (icono en la barra de tareas → "Docker Desktop is running").

1. Reconstruye y levanta los contenedores con las imágenes v3:

```
# Leer la API Key desde el archivo .env
$env:GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" |
ForEach-Object { $_.ToString().Split("=",2)[1] })
docker-compose up --build -d
```

2. Envía una consulta de prueba al frontend (puerto 80):

```
Invoke-WebRequest "http://localhost:80/generate?
prompt=Test_Communication_Logs"
```

3. Verifica la traza de logs de comunicación a través del endpoint `/logs`:

```
(Invoke-WebRequest "http://localhost:80/logs").Content
```

Salida esperada (en formato JSON):

```
{
  "logs": [
    "2026-06-04 20:50:44,123 - INFO - Enviando prompt al servicio B en la
    URL http://backend:8080/generate",
    "2026-06-04 20:50:44,456 - INFO - Respuesta exitosa recibida del
    servicio B."
  ]
}
```

```
PS C:\Users\daniel\azure-assignment> docker-compose up --build -d
time="2026-06-04T22:11:41.087Z" level=warning msg="C:\Users\daniel\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Building 3.8s (28/28) FINISHED
-> [internal] load build definitions
-> -- reading from stdin 3.08s
-> [frontend internal] load build definition from Dockerfile
-> -- transferring Dockerfile: 407s
-> [backend internal] load build definition from Dockerfile
-> -- transferring Dockerfile: 508s
-> [frontend internal] load metadata for docker.io/library/python:3.11-slim
-> [backend internal] load .dockerignore
-> -- transferring content: 2s
-> [frontend internal] load .dockerignore
-> -- transferring content: 2s
-> [frontend 1/5] FROM docker.io/library/python:3.11-slim@sha256:sha256:6b64c2d0e1146d13c22093c4e49898774872fcd6c38293747e0b3
-> -- resolve docker.io/library/python:3.11-slim@sha256:sha256:6b64c2d0e1146d13c22093c4e49898774872fcd6c38293747e0b3
-> [backend internal] load build context
-> -- transferring content: 61s
-> [frontend internal] load build context
-> -- transferring content: 61s
-> CACHED [frontend 2/5] WORKDIR /app
-> CACHED [backend 1/5] COPY requirements.txt
-> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt
-> CACHED [backend 5/5] COPY app.py
-> CACHED [frontend 3/5] COPY requirements.txt
-> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt
-> CACHED [frontend 5/5] COPY app.py
-> [backend] exporting to image
-> -- exporting layers
-> -- exporting manifest sha256:0e0b0f386040f092231d0b1f7d8a7c202d1cf076c5c19313b1f4bed
-> -- exporting config sha256:fa18f9cc7e782b3a1f738600a5f4816f887789a2c0d8b3
-> -- exporting attestation manifest sha256:d8bd82187377944804b6a867a3121d4c19665e969f87176cf83ca8fc
-> -- exporting manifest list sha256:36134b79c0c4d18064679c0a4d8fc77bd1538a09754ed187763d7
-> -- using to planerviciola.azurecr.io/fastapi-backend:v3
-> -- unpacking to planerviciola.azurecr.io/fastapi-backend:v3
-> [frontend] exporting to image
-> -- exporting layers
-> -- exporting manifest sha256:9d4bdc5d93f2c242f4b2b40c4f8c8c31249810c36377218f683276ac
-> -- exporting config sha256:36134b79c0c4d18064679c0a4d8fc77bd1538a09754ed187763d7
-> -- exporting attestation manifest sha256:37c0d3b3af77f4681809a6720b1c12a5d38767607f483c3ac320a
-> -- exporting manifest list sha256:082a12719fef365225dc2f60a8f3839c4701c19458002b3836a1a488c7db
-> -- using to planerviciola.azurecr.io/fastapi-frontend:v3
-> -- unpacking to planerviciola.azurecr.io/fastapi-frontend:v3
-> [frontend] resolving provenance for metadata file
-> [backend] resolving provenance for metadata file
[*] Parsing GC
# planerviciola.azurecr.io/fastapi-frontend:v3      built      0.0s
# planerviciola.azurecr.io/fastapi-backend:v3      built      0.0s
# Network azure-assignment-default                 Created    0.1s
# Volume azure-assignment-AlarmasientoPersistente  Created    0.0s
# Container azure-assignment-backend1             Started    0.0s
# Container azure-assignment-frontend1             Started    1.0s
PS C:\Users\daniel\azure-assignment> Invoke-WebRequest "http://localhost:80/generate?prompt=Test_Communication_Logs"

StatusCode      : 200
StatusDescription : OK
Content          : {"prompt":"Test Communication logs","text":"Okay, when you say \"Test Communication Logs,\" it typically refers to simulated or actual logs generated during a testing phase (e.g., unit testing, integration)."}
RawContent      : HTTP/1.1 200 OK
Date            : Thu, 04 Jun 2026 20:51:55 GMT
Content-Type     : application/json

Headers         : [{"Name": "System.String[]", "Value": "System.String[]", "Content-Type": "System.String[]"}]
Images          : {}
SuperFields     : {}
Links           : {}
RawContentLength: 4208
RelationLink     : {}

PS C:\Users\daniel\azure-assignment> (Invoke-WebRequest "http://localhost:80/logs").Content
{"logs":["2026-06-04 20:51:55,684 - INFO - Enviando prompt al servicio B en la URL http://backend:8080/generate","2026-06-04 20:52:17,884 - INFO - HTTP Request: GET http://backend:8080/generate?prompt=Test_Communication_Logs [\"HTTP/1.1 200 OK\", \"2026-06-04 20:52:17,884 - INFO - Respuesta exitosa recibida del servicio B.\"]"]}
PS C:\Users\daniel\azure-assignment>
```

4. Detén y elimina por completo los contenedores para validar la persistencia:

```
docker-compose down
```

5. Vuelve a iniciar los contenedores (esta vez sin reconstruir la imagen):

```
docker-compose up -d
```

6. Consulta nuevamente el endpoint de logs:

```
(Invoke-WebRequest "http://localhost:80/logs" -UseBasicParsing).Content
```

Resultado: El endpoint debe seguir devolviendo los registros de logs previos ("INFO - Enviando prompt..." y "INFO - Respuesta exitosa..."), demostrando que el archivo se almacena en el volumen persistente **AlmacenamientoPersistente** en tu equipo local y sobrevive a la recreación de los contenedores.

```
PS C:\Users\danie\azure-assignment> docker-compose down
time="2026-06-04T22:37:20+02:00" level=warning msg="C:\\Users\\danie\\azure-assignment\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
  ✓ Container azure-assignment-frontend-1 Removed                                0.6s
  ✓ Container azure-assignment-backend-1 Removed                               0.7s
  ✓ Network azure-assignment_default Removed                                   0.4s
PS C:\Users\danie\azure-assignment> docker-compose up -d
time="2026-06-04T22:37:30+02:00" level=warning msg="C:\\Users\\danie\\azure-assignment\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
  ✓ Network azure-assignment_default Created                                    0.1s
  ✓ Container azure-assignment-backend-1 Started                               0.6s
  ✓ Container azure-assignment-frontend-1 Started                              0.8s
PS C:\Users\danie\azure-assignment> (Invoke-WebRequest "http://localhost:80/logs").Content
{"logs":["2026-06-04 20:31:55,684 - INFO - Enviando prompt al servicio B en la URL http://backend:8080/generate","2026-06-04 20:32:17,984 - INFO - HTTP Request: GET http://backend:8080/generate?prompt=Test_Communication_Logs `\\HTTP/1.1 200 OK`","2026-06-04 20:32:17,984 - INFO - Respuesta exitosa recibida del servicio B."]}
PS C:\Users\danie\azure-assignment>
```

4.5. Instrucciones de Despliegue en Azure (v3)

Para desplegar esta nueva configuración (imágenes **v3** y persistencia en el Frontend) en Azure, se asume que ya se ha provisionado la infraestructura base en los bloques previos. En concreto, debes contar con:

- El **Grupo de Recursos** (**gruporecursosia**), el **Azure Container Registry** (ACR, **planservicioia**) y el **Plan de App Service** (**plan-fastapi-gemini**), cuyos comandos de aprovisionamiento se detallan en la [Sección 1.4 \(Paso 3 y Paso 6\)](#).
- La **Cuenta de Almacenamiento** (**almacenamientoappia**) y el **File Share** (**archivos-texto**), creados en la [Sección 3.1 \(Paso 2\)](#).
- Haber iniciado sesión en Azure mediante CLI (**az login**), como se indica en la [Sección 1.4 \(Paso 1\)](#).

Si cumples con estos prerrequisitos, ejecuta los siguientes comandos en tu terminal de PowerShell:

A. Construcción y subida de las nuevas imágenes v3

```
# Exportamos el nombre del ACR para etiquetar las imágenes v3
$env:ACR_NAME=$ACR_NAME

# Autenticarse en el ACR antes de hacer push (usando password-stdin por seguridad)
echo $ACR_PASSWORD | docker login "$ACR_NAME.azurecr.io" --username $ACR_NAME --password-stdin

# Construimos y subimos las imágenes
docker-compose build
docker-compose push
```

```

PS C:\Users\danie\azure-assignment> $env:ACR_NAME=$ACR_NAME
>>
PS C:\Users\danie\azure-assignment> docker-compose build
>>
time="2026-06-04T22:41:40+02:00" level=warning msg="C:\Users\danie\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Building 2/3s (20/20) FINISHED
-> [internal] load local bake definitions                                0.0s
-> -> reading from stdin 1.08s                                           0.0s
-> [backend internal] load build definition from Dockerfile              0.0s
-> -> transferring dockerfile: 508B                                       0.0s
-> [frontend internal] load build definition from Dockerfile            0.0s
-> -> transferring dockerfile: 487B                                        0.0s
-> [backend internal] load metadata for docker.io/library/python:3.11-slim 1.0s
-> [backend internal] load .dockerignore                                 0.0s
-> -> transferring context: 2B                                             0.0s
-> [frontend internal] load .dockerignore                                0.0s
-> -> transferring context: 2B                                             0.0s
-> [frontend 1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a833c22093cb840908979487a9fc0e6e38295747e49ac9 0.0s
-> -> resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a833c22093cb840908979487a9fc0e6e38295747e49ac9 0.0s
-> [backend internal] load build context                                0.0s
-> -> transferring context: 63B                                            0.0s
-> [frontend internal] load build context                                0.0s
-> -> transferring context: 63B                                            0.0s
-> CACHED [frontend 2/5] WORKDIR /app                                    0.0s
-> CACHED [backend 3/5] COPY requirements.txt .                           0.0s
-> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [backend 5/5] COPY app.py .                                     0.0s
-> CACHED [frontend 3/5] COPY requirements.txt .                         0.0s
-> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [frontend 5/5] COPY app.py .                                    0.0s
-> [frontend] exporting to image                                         0.2s
-> -> exporting layers                                                    0.0s
-> -> exporting manifest sha256:9d4abc5d6f3f2c942f4b62b469caf44c9ec1824981bcb83f77a18f0e83276bac 0.0s
-> -> exporting config sha256:9b18d7854f0623b44b4fdcfabe9e10843b2b55834be4a82c342a4a8c7e19cf4 0.0s
-> -> exporting attestation manifest sha256:b39c58f80c28efc34aafdb9752772e8f192664887f2da7c6124657cc282fcd9 0.1s
-> -> exporting manifest list sha256:1ffb80f63f486c443d82b70f018984156bf3377a9887f34f1a85e8cedab4 0.0s
-> -> naming to planservicioia.azurecr.io/fastapi-frontend:v3          0.0s
-> -> unpacking to planservicioia.azurecr.io/fastapi-frontend:v3       0.0s
-> [backend] exporting to image                                         0.2s
-> -> exporting layers                                                    0.0s
-> -> exporting manifest sha256:0849673b634074092251ed3be1f768ba7ac2852d27e6785c639533be1f4ded 0.0s
-> -> exporting config sha256:faf6f9fcec97c82ecf58c2c1b41f2d6c698adaf4de1af6f88775495cec8cd53 0.0s
-> -> exporting attestation manifest sha256:c87b7dd204c8317f2a093f00667089ff8213978c2240870c83a1f0eb8824b5e 0.1s
-> -> exporting manifest list sha256:3fa121a3f5b14aa6dfffa18a6a85c29d0e35cd5d1695a52c39eac1cc7c0513963 0.0s
-> -> naming to planservicioia.azurecr.io/fastapi-backend:v3          0.0s
-> -> unpacking to planservicioia.azurecr.io/fastapi-backend:v3       0.0s
-> [frontend] resolving provenance for metadata file                    0.0s
-> [backend] resolving provenance for metadata file                    0.0s
[*] Building 2/2
✓ planservicioia.azurecr.io/fastapi-frontend:v3 Built 0.0s
✓ planservicioia.azurecr.io/fastapi-backend:v3 Built 0.0s
PS C:\Users\danie\azure-assignment> docker-compose push
>>
time="2026-06-04T22:41:48+02:00" level=warning msg="C:\Users\danie\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Pushing 18/18
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: b8278f1c19f3 Pushed 0.5s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 571cd5d75da Pushed 0.5s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 797d495f2c68 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 5b4d6ff92fc4 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: b727f22bc825 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 45806c0e0a9 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 8649771fee17 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 5dbf852422f1 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 3e75f8258abc Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 5542906c1817 Pushed 0.5s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 97ff604ed1b0 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 5b4d6ff92fc4 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 797d495f2c68 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 5dbf852422f1 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 8649771fee17 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 45806c0e0a9 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: e285c1788bd9 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: bf45fb0e4797 Pushed 0.5s
PS C:\Users\danie\azure-assignment>

```

B. Actualización de la Web App en Azure

```

# Definir variables de entorno (por si inicias una nueva sesión de terminal)
$RESOURCE_GROUP = "gruporecursosia"
$ACR_NAME        = "planservicioia"
$WEB_APP_NAME    = "webapp-fastapi-gemini"
$STORAGE_ACCOUNT = "almacenamientoappia"
$SHARE_NAME      = "archivos-texto"
$ACR_PASSWORD    = (az acr credential show --name $ACR_NAME --query
"passwords[0].value" -o tsv)
$STORAGE_KEY     = (az storage account keys list --resource-group $RESOURCE_GROUP
--account-name $STORAGE_ACCOUNT --query "[0].value" --output tsv)

# Forzar a Azure a leer la nueva configuración del docker-compose.yml con las
imágenes v3
az webapp config container set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file

```

```

docker-compose.yml

# Reconectar las credenciales del ACR a la Web App (necesario tras actualizar el
compose, usando parámetros actualizados)
az webapp config container set --name $WEB_APP_NAME --resource-group
$RESOURCE_GROUP `
  --container-registry-url "https://$ACR_NAME.azurecr.io" `
  --container-registry-user $ACR_NAME `
  --container-registry-password $ACR_PASSWORD

# Leer la API Key desde el archivo .env
$GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" | ForEach-
Object { $_.ToString().Split("=",2)[1] })

# Configurar variables de entorno: API Key, puerto público y URL interna del
backend
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME `
  --settings GEMINI_API_KEY=$GEMINI_API_KEY WEBSITES_PORT=80
BACKEND_URL="http://localhost:8080"

```

Nota: Azure App Service Multi-container hace que todos los contenedores compartan el mismo espacio de red, por lo que el Frontend debe llamar al Backend mediante `http://localhost:8080` en lugar de `http://backend:8080` (nombre de servicio de Compose). Esto queda reflejado en la variable `BACKEND_URL`.

C. Configuración del Almacenamiento Persistente en Azure Dado que el volumen `AlmacenamientoPersistente` ahora se monta sobre el Frontend (para guardar `history.txt`), Azure se encargará de mapear automáticamente el File Share al contenedor Frontend gracias al ID coincidente. Para asociarlo (eliminando previamente cualquier montaje con el mismo ID si existe):

```

# 1. Eliminar un montaje previo con el mismo ID si existe
az webapp config storage-account delete --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --custom-id AlmacenamientoPersistente

# 2. (Opcional) Limpiar el historial previo en la nube para empezar con una demo
limpia (si no existe el archivo, dará error, el cual se puede ignorar)
az storage file delete --share-name $SHARE_NAME --account-name $STORAGE_ACCOUNT --
account-key $STORAGE_KEY --path history.txt

# 3. Asociar la cuenta de almacenamiento a la Web App
az webapp config storage-account add --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --custom-id AlmacenamientoPersistente --storage-type AzureFiles --
account-name $STORAGE_ACCOUNT --share-name $SHARE_NAME --access-key $STORAGE_KEY -
-mount-path /app/data

```

[illegible]

D. Reinicio y Verificación

Para comprobar la persistencia de los logs ante reinicios y recreaciones de los contenedores en la nube:

1. Realiza el reinicio inicial para aplicar la configuración de almacenamiento y descargar las nuevas imágenes (si la Web App está detenida, utiliza **start** en lugar de **restart**):

```
# Si el servicio ya está activo/corriendo:
az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP

# Si el servicio se detuvo previamente:
az webapp start --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

2. Envía una petición de generación de contenido (espera un par de minutos a que los contenedores hayan arrancado):

```
Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/generate?prompt=Azure_Prompt_v3" -UseBasicParsing
```

(Nota: Durante el periodo de arranque, es completamente normal y esperable que la petición falle inicialmente con un error de conexión `{"detail": "ConnectError('All connection attempts failed')"}` mientras el contenedor backend termina de iniciar. Espera uno o dos minutos y vuelve a realizar la llamada).

3. Comprueba los logs en el nuevo endpoint para verificar el flujo de comunicación inicial:

```
(Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/logs" -
UseBasicParsing).Content
```

4. (Opcional) Verifica que el archivo de logs se ha creado físicamente en el almacenamiento de Azure Files mediante CLI:

```
# Listar archivos en el File Share
az storage file list --share-name $SHARE_NAME --account-name
$STORAGE_ACCOUNT --account-key $STORAGE_KEY --output table

# Descargar y ver el contenido de history.txt
az storage file download --share-name $SHARE_NAME --account-name
$STORAGE_ACCOUNT --account-key $STORAGE_KEY --path history.txt --dest .
Get-Content history.txt
Remove-Item history.txt
```

5. Fuerza un reinicio de la Web App en Azure para simular la destrucción y recreación de los contenedores:

```
az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

6. Realiza de nuevo la consulta al endpoint de logs (espera el arranque):

```
(Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/logs" -
UseBasicParsing).Content
```

Resultado esperado: El endpoint debe seguir devolviendo los registros de logs previos ("INFO - Enviando prompt..." e "INFO - Respuesta exitosa..."), demostrando que el historial persistió gracias al montaje de Azure Files.

```

PS C:\Users\daniel\azure-assignment> az webapp start --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
>>
PS C:\Users\daniel\azure-assignment> Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?prompt=Azure_Prompt_v3"
Invoke-WebRequest:
{
  "detail": "ConnectError(\u0027All connection attempts failed\u0027)"
}
PS C:\Users\daniel\azure-assignment> Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?prompt=Azure_Prompt_v3"
Invoke-WebRequest:
{
  "detail": "ConnectError(\u0027All connection attempts failed\u0027)"
}
PS C:\Users\daniel\azure-assignment> Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?prompt=Azure_Prompt_v3"
>>

Status: 200
StatusDescription: OK
Content: [{"prompt": "Azure_Prompt_v3", "text": "\u0027Azure_Prompt_v3\u0027 is not an official, publicly documented Microsoft product or API version name. However, based on the pattern of \u0027Azure\u0027 (Microsoft\u0027s cloud pla..."}
RawContent: HTTP/1.1 200 OK
Date: Thu, 04 Jun 2026 20:57:30 GMT
Server: uvicorn
Set-Cookie: ARRAFFinity=279e2686d5d985a92f8e5544b67f8add363f7788a222be35d50834f8dfedba;Path=/;HttpOnly;Secure;Domain=webapp-fas...
Headers: [{"Date", System.String[]}, {"Server", System.String[]}, {"Set-Cookie", System.String[]}, {"Content-Length", System.String[]}]
Images: {}
InputFields: {}
Links: {}
RawContentLength: 4729
Content: [{"prompt": "Azure_Prompt_v3", "text": "\u0027Azure_Prompt_v3\u0027 is not an official, publicly documented Microsoft product or API version name. However, based on the pattern of \u0027Azure\u0027 (Microsoft\u0027s cloud pla..."}
RawContent: HTTP/1.1 200 OK
Date: Thu, 04 Jun 2026 20:57:30 GMT
Server: uvicorn
Set-Cookie: ARRAFFinity=279e2686d5d985a92f8e5544b67f8add363f7788a222be35d50834f8dfedba;Path=/;HttpOnly;Secure;Domain=webapp-fas...
Headers: [{"Date", System.String[]}, {"Server", System.String[]}, {"Set-Cookie", System.String[]}, {"Content-Length", System.String[]}]
Images: {}
InputFields: {}
Links: {}
RawContentLength: 4729
RelationLink: {}

PS C:\Users\daniel\azure-assignment> (Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/logs").Content
>>
{"logs": [{"Azure_Prompt_1", "Azure_Prompt_2", "2026-06-04 20:57:16,763 - INFO - Enviando prompt al servicio B en la URL http://backend:8080/generate", "2026-06-04 20:57:18,589 - ERROR - Error de comunicaci\u00f3n inesperado con servicio B: ConnectError('All connection attempts failed')", "2026-06-04 20:57:24,707 - INFO - Enviando prompt al servicio B en la URL http://backend:8080/generate", "2026-06-04 20:57:24,877 - ERROR - Error de comunicaci\u00f3n inesperado con servicio B: ConnectError('All connection attempts failed')", "2026-06-04 20:57:30,817 - INFO - Enviando prompt al servicio B en la URL http://backend:8080/generate", "2026-06-04 20:57:46,528 - INFO - HTTP Request: GET http://backend:8080/generate?prompt=Azure_Prompt_v3 \"HTTP/1.1 200 OK\"", "2026-06-04 20:57:46,539 - INFO - Respuesta exitosa recibida del servicio B."}]
PS C:\Users\daniel\azure-assignment> az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
>>
PS C:\Users\daniel\azure-assignment> (Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/logs").Content
>>
{"logs": [{"Azure_Prompt_1", "Azure_Prompt_2", "2026-06-04 20:57:16,763 - INFO - Enviando prompt al servicio B en la URL http://backend:8080/generate", "2026-06-04 20:57:18,589 - ERROR - Error de comunicaci\u00f3n inesperado con servicio B: ConnectError('All connection attempts failed')", "2026-06-04 20:57:24,707 - INFO - Enviando prompt al servicio B en la URL http://backend:8080/generate", "2026-06-04 20:57:24,877 - ERROR - Error de comunicaci\u00f3n inesperado con servicio B: ConnectError('All connection attempts failed')", "2026-06-04 20:57:30,817 - INFO - Enviando prompt al servicio B en la URL http://backend:8080/generate", "2026-06-04 20:57:46,528 - INFO - HTTP Request: GET http://backend:8080/generate?prompt=Azure_Prompt_v3 \"HTTP/1.1 200 OK\"", "2026-06-04 20:57:46,539 - INFO - Respuesta exitosa recibida del servicio B."}]
PS C:\Users\daniel\azure-assignment>

```

4.6. Control de Gastos y Limpieza de Recursos (Azure Cost Management)

Para evitar que los recursos aprovisionados sigan consumiendo saldo de tus cr\u00e9ditos de estudiante o facturando importes adicionales tras haber finalizado la pr\u00e1ctica, se aconseja gestionar la parada o eliminaci\u00f3n del entorno con uno de los siguientes m\u00e9todos:

Opci\u00f3n A: Detener la ejecuci\u00f3n de la Web App (Conserva la configuraci\u00f3n)

Si quieres pausar temporalmente el servicio (por ejemplo, para seguir haciendo pruebas en otro momento o para grabar el v\u00eddeo explicativo) deteniendo la ejecuci\u00f3n de los contenedores:

```

# Detener la Web App
az webapp stop --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP

# Reanudar la Web App
az webapp start --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP

```

⚠ Nota: Ten en cuenta que si el Plan de App Service est\u00e1 en un tier de pago (como el SKU B1 recomendado para multi-contenedor), el coste del plan de hospedaje se seguir\u00e1 cobrando por hora aunque la Web App est\u00e9 detenida.

Opci\u00f3n B: Eliminar el Grupo de Recursos por completo (Recomendado al finalizar)

Si has completado toda la pr\u00e1ctica y grabado el v\u00eddeo explicativo, la forma id\u00f3nea de detener el gasto al 100% de manera definitiva es destruir el grupo de recursos completo. Esto eliminar\u00e1 de forma permanente la Web App, el App Service Plan, el Azure Container Registry (ACR) y la Storage Account:

```
# Eliminar el grupo de recursos completo sin esperar confirmación interactiva
az group delete --name $RESOURCE_GROUP --yes --no-wait
```

Referencias y Bibliografía

Para la elaboración de este despliegue y la configuración de los comandos utilizados a lo largo de la práctica, se han consultado las siguientes fuentes de documentación oficial:

Documentación de Azure CLI (Microsoft Learn):

- [az login \(Autenticación en Azure\)](#)
- [az group \(Administración de Grupos de Recursos\)](#)
- [az acr \(Gestión de Azure Container Registry\)](#)
- [az acr credential \(Obtención de Credenciales del Registro\)](#)
- [az appservice plan \(Configuración de Planes de App Service\)](#)
- [az webapp \(Creación y Gestión de Web Apps\)](#)
- [az webapp config container \(Configuración de Contenedores en Web Apps\)](#)
- [az webapp config appsettings \(Definición de Variables de Entorno y Configuración\)](#)
- [az webapp config storage-account \(Montaje de Almacenamiento en Web Apps\)](#)
- [az webapp log \(Descarga y Seguimiento de Logs en Tiempo Real\)](#)
- [az storage account \(Gestión de Cuentas de Almacenamiento\)](#)
- [az storage share-rm \(Gestión de Recursos Compartidos de Azure Files\)](#)

Documentación de Docker:

- [Referencia de la Línea de Comandos de Docker \(build, push, run, login\)](#)
- [Guía de Referencia del Archivo Dockerfile](#)
- [Docker Compose — Referencia de la CLI \(up, down, build, push\)](#)
- [Docker Compose — Especificación del Archivo docker-compose.yml](#)

PowerShell:

- [Invoke-WebRequest \(Cmdlet para peticiones HTTP\)](#)

Librerías y SDKs:

- [Documentación Oficial de FastAPI \(Framework Web\)](#)
- [SDK de Google GenAI para Python \(Gemini 2.5\)](#)
- [httpx — Cliente HTTP asíncrono para Python](#)

Anexo: Enlaces del Proyecto

A continuación se facilitan los enlaces correspondientes al repositorio de código fuente y a la demostración del despliegue:

- **Repositorio de GitHub:** [danielquinti/azure-assignment](#)

- **Código de la primera versión (Bloque 1):** [Release v1 de GitHub](#)
- **Código de la segunda versión (Bloque 2):** [Release v1-compose de GitHub](#)
- **Código de la tercera versión (Bloque 3):** [Release v2 de GitHub](#)
- **Prueba de Despliegue (Vídeo):** [Demostración en vídeo de la práctica \(SharePoint\)](#)